

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI  
I TECHNIK INFORMACYJNYCH



Instytut Mikroelektroniki i Optoelektroniki

# Praca dyplomowa inżynierska

na kierunku Elektronika  
w specjalności Elektronika i Fotonika

Energooszczędny czujnik wilgotności dla inteligentnego domu

**Piotr Szewczyk**

Numer albumu 311105

promotor

dr hab. inż. Wojciech Zabołotny, prof. uczelni

WARSZAWA 2025



## **Energooszczędny czujnik wilgotności dla inteligentnego domu**

### **Streszczenie**

Celem pracy jest zaprojektowanie i zbudowanie energooszczędnego czujnika wilgotności o prostym i intuicyjnym użytkowaniu, odpowiedniego do zastosowania nie tylko w inteligentnych domach, ale także w instalacjach przemysłowych oraz innych systemach wymagających monitorowania wilgotności. Czujnik ten będzie wykorzystywany głównie do wczesnego wykrywania awarii instalacji hydraulicznych, jednakże może być również skonfigurowany jako standardowe urządzenie domowego użytku. Głównym celem projektu jest osiągnięcie minimalnego zużycia energii, co pozwoli na maksymalne wydłużenie czasu pracy urządzenia na jednej baterii. Konstrukcja kładzie nacisk na prostotę, aby użytkownik mógł bez problemu naładować lub wymienić baterię, co zwiększa praktyczność i uniwersalność urządzenia.

Czujnik składa się z kluczowych komponentów: czujnika wilgotności, nadajnika, baterii oraz mikrokontrolera. Dane z czujnika są przesyłane do minikomputera Raspberry Pi, wyposażonego w odbiornik, który przetwarza informacje i prezentuje je w systemie Home Assistant. Zastosowanie tej platformy umożliwia elastyczne zarządzanie oraz dalsze rozwijanie projektu.

Praca zawiera szczegółowy opis zaprojektowanego układu, koncentrując się na wyborze komponentów oraz implementacji oprogramowania. Omówiono również integrację z systemem Home Assistant, co umożliwia zastosowanie projektu w różnych środowiskach. Przeprowadzone testy potwierdziły zdolność urządzenia do długotrwałego działania na zasilaniu bateryjnym oraz dokładność transmisji danych do systemu.

Wyniki testów wykazują wysoką efektywność energetyczną czujnika oraz stabilność działania systemu monitorowania. Projekt spełnia wszystkie założenia, oferując łatwość użytkowania i szerokie możliwości zastosowania, co czyni go odpowiednim rozwiązaniem zarówno dla domów, jak i instalacji przemysłowych.

**Słowa kluczowe:** pomiar wilgotności, oszczędzanie energii, system monitorowania, transmisja danych, Internet Rzeczy



## **Energy-efficient humidity sensor for the smart home**

### **Abstract**

This thesis aims to design and build an energy-efficient humidity sensor with a simple and intuitive user interface, suitable for use not only in smart homes but also in industrial installations and other systems requiring humidity data monitoring. This sensor will primarily be used to predict failures in plumbing installations; however, it can be configured as a standard device available. The project's primary objective is to achieve minimal energy consumption, allowing the device to operate on a single battery for an extended period. The design emphasizes simplicity, ensuring that users can easily replace or charge the battery, enhancing the device's practicality and versatility.

The sensor comprises key components: a humidity sensor, a transmitter, a battery, and a microcontroller. Data from the sensor is transmitted to a Raspberry Pi minicomputer equipped with a receiver, which processes the information and displays it in the Home Assistant system. This platform enables flexible management and further development of the project.

The thesis provides a detailed description of the designed circuit, focusing on selecting components and implementing the software. It also discusses integrating the Home Assistant system, allowing for the project's application in various environments. Test confirmed the device's ability to operate for prolonged periods on battery power and the accuracy of data transmission to the system.

The test results demonstrate the sensor's high energy efficiency and the monitoring system's stability. The project successfully meets all objectives, offering ease of use and broad applicability, making it suitable for residential and industrial installations.

**Keywords:** humidity measurement, energy-saving, monitoring system, data transmission, Internet of Things



## Spis treści

|       |                                                                                                 |    |
|-------|-------------------------------------------------------------------------------------------------|----|
| 1     | Wstęp .....                                                                                     | 8  |
| 1.1   | Cel i założenia projektu .....                                                                  | 9  |
| 2     | Technologie Internetu Rzeczy i metody przesyłania danych w systemach inteligentnych domów ..... | 10 |
| 2.1   | Wprowadzenie do Internetu Rzeczy (IoT) .....                                                    | 10 |
| 2.2   | Z czego składają się systemy IoT? – Architektura Internetu Rzeczy .....                         | 10 |
| 2.2.1 | Elementy Internetu Rzeczy .....                                                                 | 10 |
| 2.2.2 | Warstwy architektury IoT .....                                                                  | 11 |
| 2.2.3 | Pożądane pozytywne cechy oraz wyzwania architektury IoT .....                                   | 11 |
| 2.3   | Technologie komunikacyjne stosowane w IoT .....                                                 | 12 |
| 2.3.1 | Przegląd technologii komunikacyjnych .....                                                      | 12 |
| 2.3.2 | Podsumowanie - Porównanie technologii komunikacyjnych IoT .....                                 | 14 |
| 2.3.3 | Wybór technologii komunikacyjnej: LoRa .....                                                    | 15 |
| 3     | Projekt i realizacja części sprzętowej .....                                                    | 17 |
| 3.1   | Wybór komponentów .....                                                                         | 17 |
| 3.1.1 | Wykaz elementów i analiza alternatywnych rozwiązań .....                                        | 17 |
| 3.2   | Schemat blokowy urządzenia .....                                                                | 20 |
| 3.3   | Schemat elektryczny .....                                                                       | 21 |
| 3.3.1 | Wprowadzenie .....                                                                              | 21 |
| 3.3.2 | Część zasilania .....                                                                           | 21 |
| 3.3.3 | Część cyfrowa .....                                                                             | 25 |
| 3.4   | Schemat PCB .....                                                                               | 28 |
| 4     | Oprogramowanie, Testowanie, Integracja .....                                                    | 32 |
| 4.1   | Wprowadzenie .....                                                                              | 32 |
| 4.2   | Testy modularne .....                                                                           | 33 |
| 4.2.1 | Testy czujników wilgotności .....                                                               | 33 |
| 4.2.2 | Testowanie LoRa .....                                                                           | 36 |
| 4.3   | Integracja z Home Assistant .....                                                               | 39 |
| 4.4   | Prototyp .....                                                                                  | 41 |
| 4.4.1 | Analiza problemów napotkanych podczas realizacji prototypu .....                                | 41 |
| 4.4.2 | Zaktualizowane oprogramowanie .....                                                             | 43 |
| 4.4.3 | Testy prototypu .....                                                                           | 47 |
| 4.4.4 | Możliwe optymalizacje i dalsze kroki .....                                                      | 50 |
| 5     | Podsumowanie .....                                                                              | 52 |
|       | Bibliografia .....                                                                              | 53 |

|                    |    |
|--------------------|----|
| Spis rysunków..... | 55 |
| Spis tabel.....    | 56 |





# 1 Wstęp

W dzisiejszych czasach, cechujących się szybkim rozwojem technologii, każdemu zależy na jak najbardziej komfortowym i prostym jej użytkowaniu. Życie wielu ludzi staje się coraz łatwiejsze dzięki dynamicznemu rozwojowi sektora technologicznego, jakim jest Internet Rzeczy (IoT). Pojawia się coraz więcej urządzeń i coraz większe zapotrzebowanie na urządzenia z nim związane. Łączą one w sobie niezawodność, energooszczędność oraz łatwość integracji z dotychczas powstałymi infrastrukturami. Monitorowanie wilgotności w przestrzeniach mieszkalnych lub większych konstrukcjach, jest jedną z często spotykanych funkcji takich systemów. Zapewnia ono komfort użytkownikowi, wspierając działanie systemów klimatyzacyjnych, nawilżaczy, by zapewniać warunki sprzyjające zdrowiu. Odpowiednie dane o wilgotności w trudno dostępnym miejscu w gospodarstwie domowym, mogą zapobiec poważnej awarii, a w najgorszym wypadku ochronić przed poważnym uszkodzeniem mienia.

Dzięki IoT, tworzony czujnik wilgotności, może być częścią działającego autonomicznie systemu, zbierającego i analizującego dane w czasie rzeczywistym, w którym wszystkie urządzenia podobne do niego, komunikują się ze sobą. Szczególne znaczenie mają rozwiązania energooszczędne, po których instalacji, użytkownik w zasadzie zapomina o urządzeniu, działa ono w tle, po cichu wspomagając jego życie. Urządzenia bateryjne mogące działać przez długi czas bez konieczności ładowania, są kluczowe dla praktycznego zastosowania w inteligentnych domach i budynkach.

Na rynku istnieje wiele urządzeń IoT przeznaczonych do monitorowania wilgotności, jednakże zdecydowana większość z nich ogranicza się do lokalnego przesyłania danych za pomocą technologii takich jak Wi-Fi, czy Bluetooth. Wciąż dla wielu scenariuszy są to bardzo dobre i podstawowe rozwiązania, jednakże mają swoje ograniczenia, takie jak ograniczony zasięg transmisji oraz wysokie zużycie energii. Alternatywą dla tych dwóch technologii, jest coraz to bardziej powszechna technologia LoRa (ang. Long Range), umożliwiająca przesyłanie danych na duże odległości przy minimalnym zużyciu energii.

Od standardowego rozwiązania, którym jest prosty domowy czujnik wilgotności, czujnik będący przedmiotem tej pracy różni się tym, że instaluje się go w ciężko dostępnych obszarach, np. w pobliżu rur pod wanną, za pralką czy w innych miejscach, do których użytkownik nie ma bezpośredniego dostępu. Jego czas pracy jest znacznie dłuższy, jednakże przy odpowiedniej zmianie w oprogramowaniu, może być również skonfigurowany jako zwykłe urządzenie codziennego użytku.

## 1.1 Cel i założenia projektu

Celem niniejszej pracy jest opracowanie energooszczędnego czujnika wilgotności, który będzie w stanie monitorować poziom wilgotności w miejscach trudno dostępnych i przesyłać dane użytkownikowi za pomocą technologii LoRa. Urządzenie to ma pełnić funkcję systemu wczesnego ostrzegania, umożliwiając jak najszybsze podjęcie kroków w celu zapobiegania awarii, takich jak wycieki wody czy uszkodzenia uszczelek, zanim doprowadzą one do poważnych strat materialnych.

Projekt zakłada pełną realizację urządzenia, obejmującą:

- zaprojektowanie i wykonanie płytki PCB,
- implementację oprogramowania umożliwiającego zbieranie danych z czujnika wilgotności, układu scalonego mierzącego poziom baterii i ich przesyłanie,
- integrację z systemem IoT umożliwiającym łatwy dostęp do informacji dla użytkownika końcowego,
- testowanie urządzenia w warunkach symulujących rzeczywiste zastosowania, np. w łazienkach, kuchniach i pomieszczeniach technicznych.

Praca ta nie tylko przyczynia się do zwiększenia bezpieczeństwa i funkcjonalności inteligentnych domów, ale także promuje zrównoważony rozwój, kładąc nacisk na energooszczędność i długotrwałość działania urządzeń IoT.

## **2 Technologie Internetu Rzeczy i metody przesyłania danych w systemach inteligentnych domów**

### **2.1 Wprowadzenie do Internetu Rzeczy (IoT)**

Pojęcie Internetu Rzeczy (ang. Internet of Things, IoT) odnosi się do urządzeń połączonych w sieci, które komunikując ze sobą nawzajem i z innymi systemami, umożliwiają przesyłanie, analizowanie i wykorzystywanie danych w czasie rzeczywistym. Internet Rzeczy w swoim zakresie obejmuje bardzo dużą ilość różnorodnych urządzeń, od czujników środowiskowych, przez inteligentne urządzenia domowe, aż po zaawansowane systemy przemysłowe.

Internet Rzeczy ułatwia, wręcz rewolucjonizuje życie codzienne wszystkich, którzy z niego korzystają. Jest w stanie zautomatyzować wiele procesów, od prostych zadań takich jak zarządzanie światłem w pomieszczeniu, po naprawdę zaawansowane takiej jak monitorowanie zdrowia lub zarządzanie ogromnymi infrastrukturami, taki jak inteligentne budynki, a nawet miasta. Wraz z rosnącą świadomością korzyści płynących z IoT, jego znaczenie dynamicznie wzrasta, co bezpośrednio napędza rozwój technologii.

### **2.2 Z czego składają się systemy IoT? – Architektura Internetu Rzeczy**

#### **2.2.1 Elementy Internetu Rzeczy**

Systemy te składają się z pewnych kluczowych elementów, które razem tworzą jedną wspólną całość komunikując się ze sobą.

- Urządzenia brzegowe, to między innymi czujniki, które zbierają dane z otoczenia. Mogą być wyposażone w np. mikrokontrolery przetwarzają dane i przygotowują je do wysłania.
- Sieci komunikacyjne za pomocą dane z urządzeń brzegowych transportowane są do serwera, sieć działa jako pośrednik między urządzeniami, a systemami centralnymi.
- Chmura obliczeniowa stanowiąca miejsce przechowywania i przetwarzania danych, do których użytkownik ma dostęp w każdym momencie.
- Aplikacje użytkownika końcowego, mobilne, czy też webowe, które umożliwiają sterowanie urządzeniami, monitoring bądź też analizę danych.

### 2.2.2 Warstwy architektury IoT

W praktyce można zauważyć wiele różnych schematów do architektury Internetu Rzeczy. Największe informatyczne i elektroniczne przedsiębiorstwa, takie jak Cisco System, Inc., Oracle Corporation, czy IEEE sugerują własne idee do tworzenia struktur architektonicznych IoT [1]. Standardowym, najbardziej bazowym modelem wizualizującym architekturę IoT jest model trójwarstwowy, w którego skład wchodzi następujące warstwy:

- Warstwa percepcji - podstawowa warstwa fizyczna, wyposażona np. w kamery, czujniki, której zadaniem jest zbieranie danych z otoczenia.
- Warstwa sieciowa (komunikacji) – zapewnia wymianę danych pomiędzy wszystkimi komponentami systemu: urządzeniami, platformą oraz użytkownikiem, a także umożliwia komunikację między użytkownikiem a urządzeniami [2].
- Warstwa aplikacji - bezpośrednio widoczna dla użytkownika końcowego. Odpowiada za wizualizację danych i interakcję z resztą urządzeń.

Dodatkowo wyróżnia się kolejne warstwy, takie jak warstwa przetwarzania, w której to zebrane dane zostają przetwarzane i analizowane, np. do dalszych automatyzacji. Warstwa bezpieczeństwa odpowiedzialna jest ochronę przesyłanych oraz gromadzonych danych. Między innymi szyfruje dane podczas ich przesyłania, autoryzuje użytkowników oraz zapobiega atakom na sieci IoT [3]. Istnieją również warstwa biznesowa, brzegowa, zarządzania i wiele innych, gdyż, temat architektury Internetu Rzeczy jest bardzo rozległy.

### 2.2.3 Pożądane pozytywne cechy oraz wyzwania architektury IoT

Tworząc strukturę Internetu Rzeczy należy dążyć do osiągnięcia jak najlepszego wykorzystania jej atrybutów:

- Skalowalność: zdolność systemu IoT do obsługi coraz większej liczby urządzeń, użytkowników lub danych bez utraty wydajności i przy minimalnych zmianach w infrastrukturze. Oznacza to, że system zaprojektowany dla kilku urządzeń może być z łatwością rozbudowany do obsługi setek lub tysięcy czujników.
- Automatyzacja: systemy wykonują określone zadania lub podejmują decyzje bez potrzeby ingerencji użytkownika. Dzięki automatyzacji, systemy IoT stają się bardziej intuicyjne i efektywne.
- Personalizacja: systemy mogą być dostosowane do potrzeb użytkownika, dostosowane może być dosłownie wszystko za pomocą UI w aplikacjach końcowych.

Wyzwania architektury IoT:

- Bezpieczeństwo: ochrona urządzeń oraz danych przed cyberatakami np. manipulacją, czy też DDoS. Autoryzowanie dostępu wskazanych użytkowników.
- Interoperacyjność: zdolność różnych urządzeń oraz systemów do współpracy i wymianą danych między sobą, pomimo działania w różnych technologiach, standardach, bądź protokołach komunikacyjnych.
- Energooszczędność: szczególnie ważne dla urządzeń zasilanych bateryjnie, bardzo ważny aspekt IoT, często układy muszą działać przez długi okres bez wymiany

zasilania, bądź są usytuowane w miejscach ciężko dostępnych, co powoduje problem z każdorazowym dostaniem się do urządzenia.

Powyższe informacje zostały opracowane na podstawie materiałów przedstawionych w załączniku [1], który zawiera szczegółowy opis kluczowych pozytywnych cech architektury IoT.

## **2.3 Technologie komunikacyjne stosowane w IoT**

Kluczowym elementem w urządzeniach oraz systemach Internetu Rzeczy, jest komunikacja, której odpowiedni wybór, definiować będzie charakterystykę tworzonego projektu. Różnorodność, szeroka gama wyboru, pozwala na dostosowanie odpowiedniego rodzaju technologii komunikacyjnej do konkretnego zastosowania np. w inteligentnych budynkach, fabrykach, czy w rolnictwie. Wybór technologii zależy od wielu czynników, takich jak zasięg, przepustowość, zużycie energii, lokalizacja.

### **2.3.1 Przegląd technologii komunikacyjnych**

Jedną z najczęściej stosowanych technologii jest Wi-Fi, znane z szerokiego zastosowania w domach, biurach i przestrzeniach publicznych. Bazując na standardach IEEE 802.11 [4], technologia ta oferuje wysoką przepustowość, sięgającą od kilku do kilkuset Mbps, oraz bardzo dobrze rozwiniętą infrastrukturę, wspieraną przez ogromną liczbę urządzeń. Dzięki bezpośredniemu połączeniu z Internetem, Wi-Fi nie wymaga dodatkowych bramek, co ułatwia integrację. Niemniej jednak, ograniczony zasięg (do 100 metrów w otwartej przestrzeni) oraz wysokie zużycie energii wykluczają jego zastosowanie w urządzeniach bateryjnych.

Bluetooth, oparty na standardzie IEEE 802.15.1 [4], jest kolejną popularną technologią komunikacji bliskiego zasięgu. Wersja BLE (Bluetooth Low Energy) została opracowana specjalnie z myślą o urządzeniach IoT, charakteryzując się niskim poborem energii oraz łatwą integracją z urządzeniami mobilnymi i systemami operacyjnymi, takimi jak Android czy iOS. Bluetooth znajduje zastosowanie w urządzeniach noszonych oraz lokalnej kontroli inteligentnych urządzeń, takich jak żarówki czy zamki drzwi. Jednakże ograniczony zasięg (do kilkunastu metrów) oraz stosunkowo niska przepustowość sprawiają, że nie nadaje się on do bardziej wymagających aplikacji.

Dla rozwiązań o średnim poborze prądu i rozszerzalnym zasięgu często wybierana jest technologia ZigBee, oparta na standardzie IEEE 802.15.4 [4]. Dzięki możliwości tworzenia sieci o architekturze mesh (siatki), ZigBee pozwala na łatwe rozszerzanie zasięgu, co sprawdza się w inteligentnych domach, systemach monitoringu czy automatyce budynkowej. Technologia ta charakteryzuje się dobrą odpornością na zakłócenia oraz średnim zużyciem energii, urządzenia pozostają w ciągłym stanie nasłuchu. Wymaga zastosowania bramki do komunikacji z Internetem i oferuje stosunkowo niską przepustowość.

LoRa, wykorzystująca modulację CSS (Chirp Spread Spectrum), to rozwiązanie przeznaczone do komunikacji dalekiego zasięgu. Działa na nielicencjonowanych pasmach częstotliwości i oferuje zasięg do kilkunastu kilometrów w otwartej przestrzeni, zachowując bardzo niski pobór energii. Technologia ta ma zastosowanie w systemach monitorowania środowiska, inteligentnych miernikach, czy nadajnikach. Jej głównym ograniczeniem jest bardzo niska

przepustowość oraz potencjalne zakłócenia wynikające z wykorzystania nielicencjonowanych pasm.

NB-IoT, rozwinięta w ramach sieci LTE, została zaprojektowana z myślą o urządzeniach IoT o niskim poborze energii. Technologia ta zapewnia stabilne połączenie w obszarach dużego obciążenia sieci, obsługując dużą liczbę urządzeń na niewielkim obszarze. Z kolei 5G IoT, będąca najnowszą technologią sieci mobilnych, wyróżnia się bardzo wysoką przepustowością (do kilku Gbps) i niskimi opóźnieniami, co czyni ją idealnym wyborem do aplikacji krytycznych, takich jak autonomiczne pojazdy czy przemysłowe systemy IoT. Mimo swoich zalet, zarówno NB-IoT, jak i 5G wymagają dostępu do zaawansowanej infrastruktury sieciowej, co wiąże się z wysokimi kosztami implementacji [5][6].

### 2.3.2 Podsumowanie - Porównanie technologii komunikacyjnych IoT

Tabela 1 Porównanie technologii komunikacyjnych IoT

| Technologia | Pasmo częstotliwości      | Zasięg        | Przepustowość | Zużycie energii | Przykłady zastosowania                                |
|-------------|---------------------------|---------------|---------------|-----------------|-------------------------------------------------------|
| Wi-Fi       | 2,4 GHz & 5 GHz           | Krótki        | Wysoka        | Wysokie         | Urządzenia wymagające dużej przepustowości danych.    |
| Bluetooth   | 2,4 GHz                   | Krótki        | Średnia       | Niskie          | Urządzenia typu wearables.                            |
| ZigBee      | 2,4 GHz                   | średni        | Niska         | Średnie         | Sieci mesh w inteligentnych domach.                   |
| LoRa        | 433 MHz, 868 MHz, 915 MHz | Bardzo długi  | Bardzo niska  | Bardzo niskie   | Monitorowanie środowiska, inteligentne domy.          |
| NB-IoT      | LTE                       | Średni/długi  | Niska         | Bardzo niskie   | Monitorowanie infrastruktury, wodomierze, gazomierze. |
| 5G IoT      | Sub-1 GHz to mmWave       | Bardzo krótki | Bardzo wysoka | Bardzo wysokie  | Systemy przemysłowe.                                  |

Analiza różnych technologii komunikacyjnych wykorzystywanych w systemach IoT wykazała, że każda z nich ma swoje unikatowe cechy i obszary zastosowań. Wybór odpowiedniej technologii zależy od specyficznych wymagań projektu, takich jak zasięg, energooszczędność, przepustowość danych oraz koszty implementacji. Technologie takie jak Wi-Fi czy Bluetooth sprawdzają się w lokalnych systemach, gdzie kluczowe są wysoka przepustowość i niski koszt. Z kolei technologie typu ZigBee, LoRa czy NB-IoT znajdują zastosowanie w systemach o większym zasięgu i ograniczonym zużyciu energii, szczególnie w przypadku urządzeń zasilanych bateryjnie.



Porównanie pokazuje, że żadne rozwiązanie nie jest uniwersalne – każde ma swoje zalety i ograniczenia, które należy wziąć pod uwagę podczas projektowania systemu IoT. W przypadku projektu energooszczędnego czujnika wilgotności, technologia LoRa wyróżnia się swoją zdolnością do pracy na dużych odległościach, co w naturalny sposób obejmuje także przesyłanie danych w budynkach, przy minimalnym zużyciu energii i wysokiej odporności na przeszkody, takie jak ściany czy rury.

Dalsza część pracy skupi się na szczegółowym uzasadnieniu wyboru technologii LoRa dla tego projektu oraz opisanu sposobu jej implementacji w zaprojektowanym systemie.

### 2.3.3 Wybór technologii komunikacyjnej: LoRa

Dlaczego właśnie LoRa?

Wybór technologii biorącej swoją nazwę (ang. Long Range), jest uzasadniona analizą potrzeb urządzenia oraz wcześniejszym porównaniem z innymi dostępnymi technologiami komunikacyjnymi Internetu Rzeczy. Kluczowym elementem była niezawodna transmisja danych z trudno dostępnych miejsc w inteligentnym domu, takich jak przestrzenie za pralką, pod wanną, czy w ścianach, bądź sufitach, przy zachowaniu energooszczędności i długiej żywotności urządzenia na zasilaniu bateryjnym. Technologia Long Range wyróżnia się idealnym połączeniem dużego zasięgu, minimalnego użycia energii i odporności na zakłócenia dopasowanym do projektu czujnika.

Sposób działania LoRa:

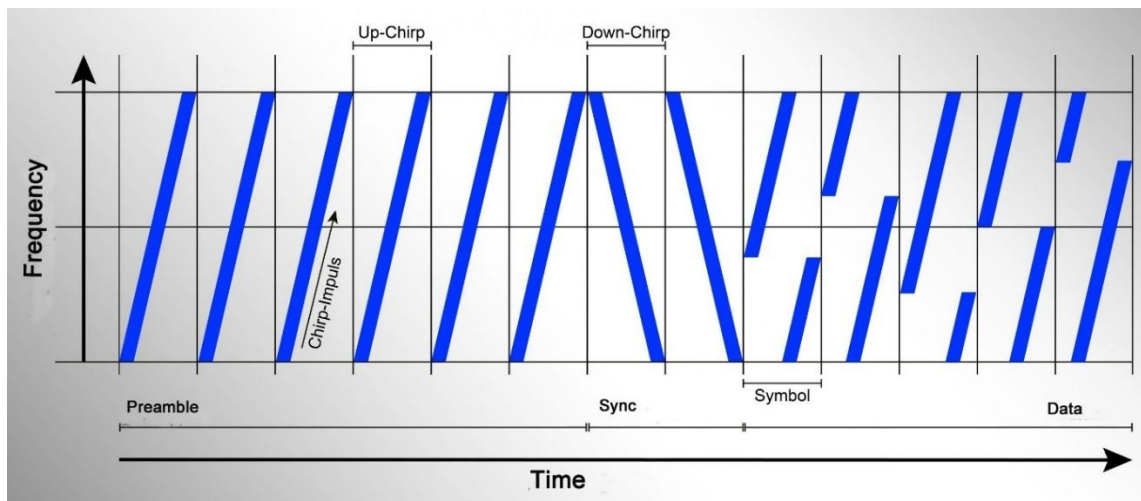
Technologia ta wykorzystuje modulację CSS (ang. Chirp Spread Spectrum). Znana jest dużej odporności na zakłócenia i zwiększenia czułości odbiornika. Użycie jej, pozwala na wysyłanie, na duże odległości sygnałów o niskiej mocy, zachowując w tym samym czasie stabilność połączenia nawet w trudnodostępnych miejscach. Jej kluczowe aspekty to:

- Modulacja CSS: wyżej wspomniana, zapewnia transmisję, nawet w trudnych warunkach lokalizacyjnych, przy niskim SNR (ang. Signal-to-noise ratio).
- Pasmo częstotliwości ISM (ang. Industrial, Scientific, and Medical): LoRa do transmisji danych wykorzystuje pasma nielicencjonowane takie jak 868 MHz, co pozwala na swobodę we wdrażaniu i kosztach implementacji.
- Tryby pracy: Możliwość zmiany parametrów takich jak moc nadawania, czy szybkość przesyłania, co pozwala na prostą optymalizację projektu.

Korzyści, wynikające z decyzji o wyborze technologii Long Range:

- Długi zasięg transmisji:

W zamkniętym terenie, z wieloma przeszkodami, pozwala wysyłać sygnał na zasięg kilku kilometrów, co w pełni pokrywa żadaną funkcjonalność czujnika. Nie trzeba używać żadnego, dodatkowego wzmacniacza sygnału. LoRa jest odporna na transmisję wielodrogową, wywoływaną przez odbicia od przeszkód sygnału radiowego. Dzięki modulacji CSS sygnał jest rozciągany na szerokie pasmo częstotliwości i każdy bit jest reprezentowany jako zmiana częstotliwości w czasie w formie „ćwierkania” (ang. chirp). Dzięki temu fale nie interferują ze sobą [7].



Rysunek 1 Zobrazowanie działania CSS [7]

- Penetracja sygnału:

Wybrane pasmo 868 MHz cechuje się bardzo dobrą zdolnością do przenikania przez przeszkody, co czyni je idealnym wyborem do zastosowań wewnątrz budynków. Sygnał nie będzie miał problemów w takich przestrzeniach jak za pralką lub wanną. Pasma 868 MHz nie jest wybrane przypadkowo. Jest ono obok pasma 433 MHz nielicencjonowanym pasmem ISM, zgodnie z regulacjami ETSI (ang. European Telecommunications Standards Institute) [8]. Co za tym idzie, użytkowanie tej częstotliwości nie wymaga żadnych dodatkowych opłat. Kolejnym nielicencjonowanym pasmem ISM jest pasmo 915 MHz, jest ono niemalże identyczne jak 868 MHz, jednakże używa się go w Stanach Zjednoczonych. Więc dlaczego wybór padł akurat na pasmo 868 MHz? Zapewnia on optymalny kompromis między zasięgiem, a przepustowością. Zapewnia wyższą przepustowość niż 433 MHz, kosztem mniejszego zasięgu. Pasma 868 MHz zapewnia również lepszą penetrację sygnału niż 433 MHz oraz stabilność, z racji tego, że drugie pasmo jest o wiele częściej używane. Do urządzeń działających na paśmie 433 MHz, potrzebna jest około 2 razy większa antena, niż w przypadku 868 MHz. Pasma 915 MHz zostaje automatycznie wykluczone ze względu na lokalizację tworzenia projektu, czyli Polskę w Europie.

- Energooszczędność:

Cała technologia LoRa została zaprojektowana z myślą o efektywnym wykorzystaniu energii w urządzeniach baterijnych. Umożliwia wprowadzenie urządzenia w tryb uśpienia między transmisjami, co pozwala na oszczędność energii. Warto jednak pamiętać, że np. urządzenia stale nasłuchujące sygnałów mogą zużywać baterię znacznie szybciej. Dlatego, oprócz wykorzystania samego wyboru technologii LoRa, kluczowe jest także zoptymalizowanie oprogramowania takich urządzeń, aby maksymalnie wydłużyć ich czas pracy.

Podsumowując technologia LoRa działająca na paśmie 868 MHz oferuje unikalne połączenie dużego zasięgu, energooszczędności i niezawodności, czyniąc ją idealnym rozwiązaniem do tworzonego projektu czujnika wilgotności. Dzięki możliwości pracy w trudnych warunkach, takich jak zamknięte przestrzenie wewnątrz budynków, oraz minimalnym kosztom implementacji, LoRa spełnia wszystkie wymagania związane z niezawodną transmisją danych w inteligentnych domach.

## 3 Projekt i realizacja części sprzętowej

### 3.1 Wybór komponentów

Przy wyborze komponentów kluczowe było spełnienie następujących założeń:

**Energooszczędność:** Każdy element musiał zużywać minimalną ilość energii, aby zapewnić jak najdłuższy czas pracy urządzenia na zasilaniu bateryjnym.

**Precyzja i niezawodność:** Wysoka dokładność pomiarów wilgotności była niezbędna do przewidywania potencjalnych awarii w instalacjach hydraulicznych.

**Kompatybilność z technologią LoRa:** Urządzenie miało komunikować się na dużą odległość przy minimalnym zużyciu energii.

**Cena:** Całkowity koszt urządzenia na potrzeby rynku powinien być jak najmniejszy.

**Mała masa i niewielkie wymiary:** Konstrukcja musiała być lekka i niewielka, aby umożliwić montaż w trudno dostępnych miejscach.

Każdy z elementów został starannie dobrany, aby w pełni odpowiadać określonym wymaganiom projektu. W kolejnej części znajduje się szczegółowy opis poszczególnych komponentów wraz z uzasadnieniem, dlaczego właśnie te rozwiązania okazały się najbardziej optymalne.

#### 3.1.1 Wykaz elementów i analiza alternatywnych rozwiązań

- Mikrokontroler: STM32L071KBU6 [9]

Na rynku dostępna jest szeroka gama mikrokontrolerów dedykowanych aplikacjom energooszczędnym, w tym rozwiązania oferowane przez renomowanych producentów, takich jak Texas Instruments (seria MSP430 [10]), czy Microchip (SAM L [11]). Wybór natomiast padł na mikrokontroler STM32L071KBU6 bazujący na energooszczędnym rdzeniu ARM Cortex-M0+, co czyni go idealnym dla aplikacji bateryjnych. STMicroelectronics NV jest jednym z najbardziej rozpoznawalnych dostawców mikrokontrolerów. Firma oferuje również bardzo czytelne i komfortowe środowisko do programowania, swoich urządzeń – CubeIDE. Dzięki niskim wartościom poboru mocy mikrokontroler dobrze pasuje do wymagań projektu. Dodatkowo szeroka gama interfejsów komunikacyjnych, takich jak I<sup>2</sup>C, SPI, czy UART, umożliwia łatwą integrację z pozostałymi elementami urządzenia. Ultra-energooszczędny mikrokontroler zarządza całym systemem, przetwarza dane z jego czujnika wilgotności i miernika baterii oraz steruje komunikacją z modułem Lora. Jego parametry poboru prądu pozwalają na wydłużenie czasu pracy urządzenia na zasilaniu bateryjnym.

- Moduł komunikacyjny LoRa: RFM95 [12]

Moduł RFM95 dostarczany przez firmę HOPERF wykorzystuje technologię LoRa, umożliwiając komunikację na dużą odległość przy minimalnym zużyciu energii. Jego czułość odbiornika na poziomie -148 dBm oraz moc nadawania do 20 dBm zapewniają stabilność i niezawodność transmisji nawet w trudnych warunkach propagacyjnych. Moduł odpowiada za transmisję danych z urządzenia do odbiornika, połączonego z brokerem MQTT. Dzięki dużemu zasięgowi i niskiej mocy pracy, idealnie nadaje się do aplikacji w inteligentnych domach. RFM95 jest jednym z najbardziej rozpowszechnionych transceiver-ów dostępnych do kupienia, obok np. SX1276 firmy Semtech [13]. Moduły te są do siebie bliźniaczo podobne, stanowią najlepszy wybór.

- Czujnik wilgotności i temperatury: HDC1080 [14]

HDC1080 to precyzyjny cyfrowy czujnik wilgotności i temperatury firmy Texas Instruments, przeznaczony do aplikacji o niskim poborze mocy, w trybie aktywnym pobiera maksymalnie 1,3  $\mu$ A, natomiast w trybie uśpienia 100 nA. HDC1080 zapewnia precyzję na poziomie  $\pm 2\%$  RH dla pomiarów wilgotności. Dodatkowo można zaprogramować zbieranie informacji o temperaturze, co zostało zaimplementowane w projekcie, gdyż kosztuje to bardzo mało dodatkowej energii. Te parametry pozwalają na monitorowanie warunków środowiskowych z dużą dokładnością, co jest kluczowe w zastosowaniach IoT, takich jak wczesne wykrywanie zmian wilgotności w trudno dostępnych miejscach. Na rynku dostępne są inne precyzyjne czujniki wilgotności, takie jak SHT3x [15], AM2320 [16] czy HTU21D(F) [17], to HDC1080 wychodzi najkorzystniej cenowo i energooszczędnie. Dobrą ideą byłoby stworzenie swojego własnego czujnika wilgotności np. metodą rezystancyjną (umieszczenie dwóch elektrod w niewielkiej odległości od siebie i połączenia ich rezystorem). Takie rozwiązanie pozwala na lepszą kalibrację urządzenia, jednakże łatwa dostępność i bardzo niski koszt gotowego rozwiązania jest o wiele bardziej wygodne i wystarczające w tworzonym projekcie.

- Układ zarządzania zasilaniem (ang. Power Path Managment): bq24078 [18]

Komponent bq24078 firmy Texas Instruments to zaawansowane rozwiązanie dedykowane zarządzaniu zasilaniem i ładowaniem baterii litowo-jonowych w urządzeniach zasilanych bateryjnie. Jednym z kluczowych atutów układu jest możliwość obsługi szerokiego zakresu napięć wejściowych, co pozwala na zasilanie go standardowym zasilaczem przez port USB-C. Komponent ten pozwala na działanie układu, zarówno podczas zasilania jak i podczas pracy bateryjnej. Układ wyróżnia się także rozbudowanymi funkcjami ochronnymi, takimi jak ochrona przed przepięciami, zwarciami czy przegrzaniem. Atutem bq24078 jest jego niski pobór mocy. W trybie uśpienia układ zużywa jedynie 10-15  $\mu$ A. Alternatywną opcją, byłoby zasilanie układu tylko za pomocą baterii, jednakże wtedy użytkownik musiałby ładować baterię dedykowaną do baterii Li-Ion zasilaczem lub kupować nową baterię, co wpłynęłoby na komfort jak i przeznaczenie dodatkowych środków pieniężnych.

- Układ monitorujący poziom naładowania (ang. fuel gauge): bq27411 [19]

bq27411 to precyzyjny monitor stanu naładowania baterii (SoC, ang. State of Charge), zaprojektowany dla baterii litowo-jonowych i litowo-polimerowych. Element dostarczany przez Texas Instruments wyróżnia się bardzo niskim poborem prądu: 93  $\mu\text{A}$  w trybie normalnym, 21  $\mu\text{A}$  w trybie uśpienia i zaledwie 9  $\mu\text{A}$  w trybie hibernacji, co czyni go idealnym do zastosowań w urządzeniach IoT zasilanych bateryjnie. Alternatywą dla bq27411 byłoby wykorzystanie ADC mikrokontrolera w połączeniu z dzielnikiem napięcia sterowanym tranzystorami MOSFET. Choć to podejście pozwala na podstawowe monitorowanie napięcia baterii i ograniczenie strat energii przez odłączanie dzielnika w czasie nieaktywnym, wciąż byłoby mniej wydajne niż bq27411. Ten gwarantuje tanie, zaawansowane i gotowe do użycia rozwiązanie, które znacznie przewyższa alternatywy, zapewniając długą żywotność baterii i najwyższą wydajność energetyczną.

- Regulator napięcia: TPS7A02 [20]

TPS7A02 to ultra-energooszczędny regulator LDO (ang. low-dropout), dostarczający stabilne napięcie zasilania przy minimalnym poborze mocy. Dzięki wyjątkowo niskiemu prądowi w trybie uśpienia, wynoszącemu zaledwie 25 nA, jest on idealnym wyborem do urządzeń IoT. Regulator obsługuje napięcia wejściowe w zakresie od 1,5 V do 6,0 V, dostarczając napięcie wyjściowe od 0,8 V do 5,0 V, z maksymalnym prądem wyjściowym wynoszącym 200 mA. Wysoka stabilność napięcia wyjściowego oraz niski poziom szumów sprawiają, że TPS7A02 doskonale nadaje się do zasilania wrażliwych układów, takich jak mikrokontrolery i moduły komunikacyjne. Możliwą alternatywą byłoby zastosowanie regulatora impulsowego, jednakże regulator LDO w porównaniu do regulatora impulsowego nie wymaga żadnych elementów zewnętrznych (takich jak cewka, tranzystory czy dioda Schottky'ego), co ułatwia projektowanie PCB. Regulator liniowy jest zdecydowanie bardziej stabilniejszy, generuje znacznie mniej zakłóceń niż regulatory impulsowe, zatem jest bardziej kompatybilny z cyfrowym HDC1080. Regulatory impulsowe cechują się wyższą sprawnością, a co za tym idzie, są bardziej przydatne w rozwiązaniach bateryjnych, ale w przypadku tworzonego projektu, gdzie obciążenie układu jest bardzo małe, a dobrany regulator LDO ma bardzo dobre parametry, różnice te są znikome, toteż postawiono na o wiele prostsze rozwiązanie.

- Antena 868 MHz: VG868SNX18-5W2

Zwykła, kompaktowa antena sprężynowa zoptymalizowana dla pasma 868 MHz  $\pm$  20 MHz, z impedancją 50  $\Omega$  i zyskiem 2 dBi. Jej niewielkie wymiary i lekka konstrukcja sprawiają, że doskonale nadaje się do urządzeń IoT wymagających niezawodnej komunikacji w technologii LoRa.

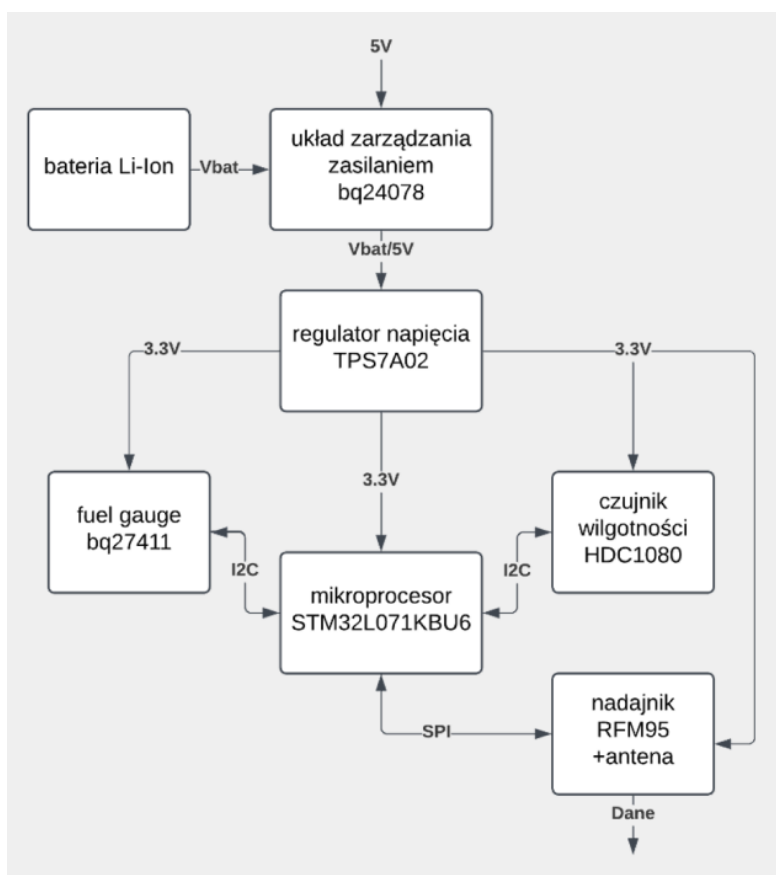
- Rezystory i kondensatory

Rezystory i kondensatory, jako podstawowe elementy pasywne, odgrywają kluczową rolę w projekcie. Są one wykorzystywane przede wszystkim do filtrowania sygnałów, stabilizacji napięcia zasilania oraz ochrony układu przed przepięciami, zapewniając niezawodną pracę urządzenia. Szczegółowy dobór wartości tych elementów oraz ich rozmieszczenie na płytce PCB zostaną omówione w dalszej części pracy, poświęconej schematowi elektrycznemu i projektowaniu układu.

### 3.2 Schemat blokowy urządzenia

Schemat blokowy urządzenia przedstawia architekturę systemu:

Czujnik wilgotności HDC1080 oraz monitor stanu naładowania baterii bq27411 komunikują się z mikrokontrolerem STM32L071KBU6 za pośrednictwem interfejsu komunikacyjnego I<sup>2</sup>C, co pozwala na szybkie i niezawodne przesyłanie danych. Mikrokontroler przetwarza zebrane informacje i przekazuje je do modułu komunikacyjnego LoRa RFM95 poprzez interfejs SPI, zapewniając płynną współpracę między komponentami. Całość układu jest zasilana przez wydajne komponenty zarządzania energią: bq24078 odpowiada za ładowanie baterii, a TPS7A02 zapewnia stabilne napięcie zasilania dla wszystkich elementów systemu.



Rysunek 2 Schemat blokowy układu

## 3.3 Schemat elektryczny

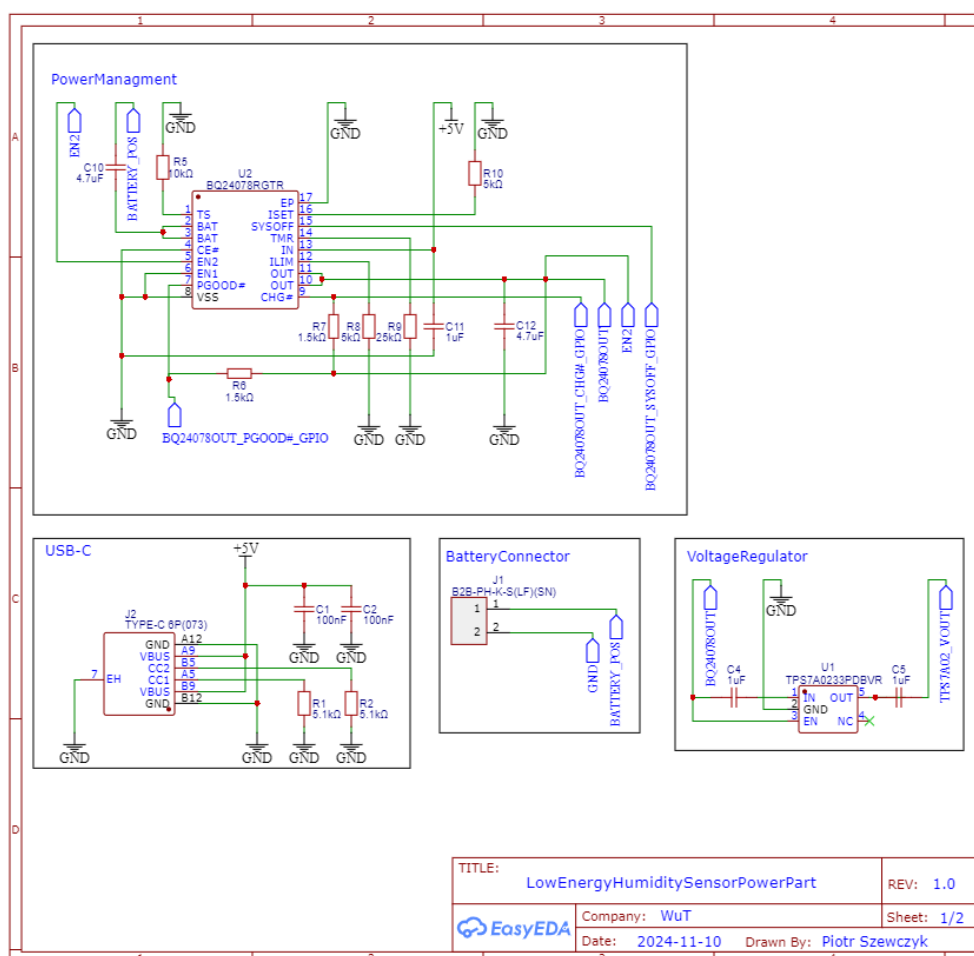
### 3.3.1 Wprowadzenie

Schemat elektryczny został zaprojektowany w darmowym, ogólnodostępnym oprogramowaniu EasyEDA, które umożliwia kompleksowe projektowanie obwodów, od schematu elektrycznego po generowanie plików produkcyjnych PCB. Całość projektu została podzielona na dwie główne części:

- Część zasilania, odpowiedzialna za ładowanie i dystrybucję energii do komponentów systemu.
- Część logiki, która obsługuje czujnik wilgotności, monitorowanie stanu baterii, mikrokontroler oraz moduł LoRa.

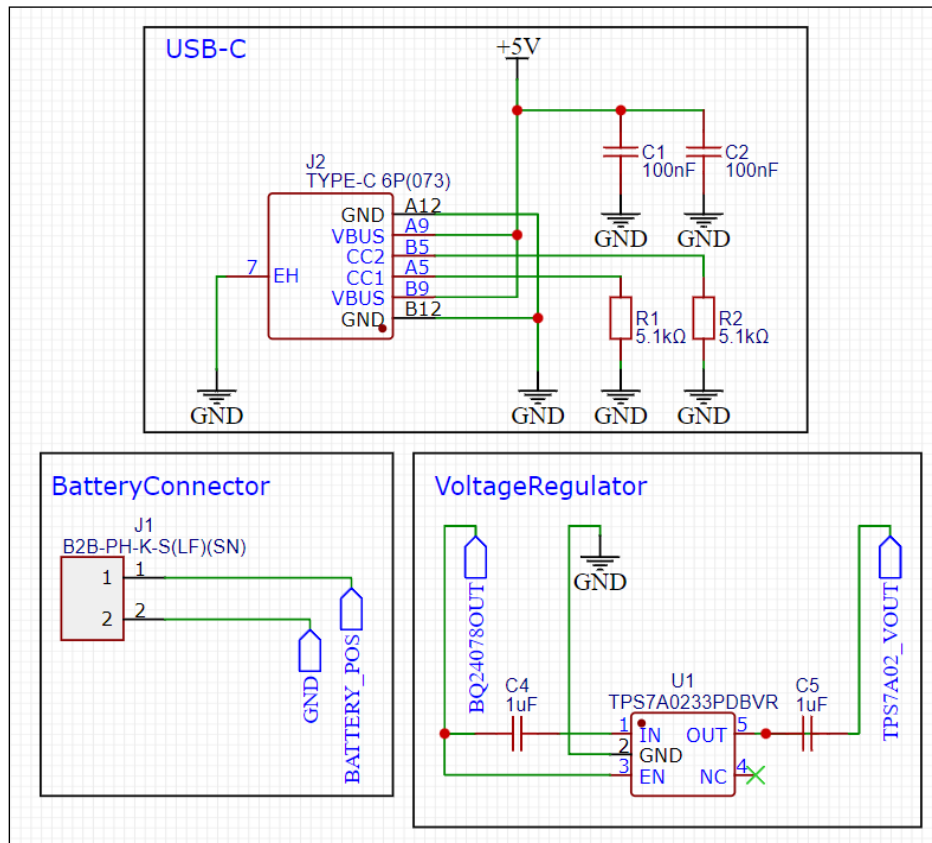
Projekt opiera się na modularnym podejściu co pozwala na łatwą diagnostykę i ewentualne modyfikacje w przyszłości.

### 3.3.2 Część zasilania



Rysunek 3 Część zasilania

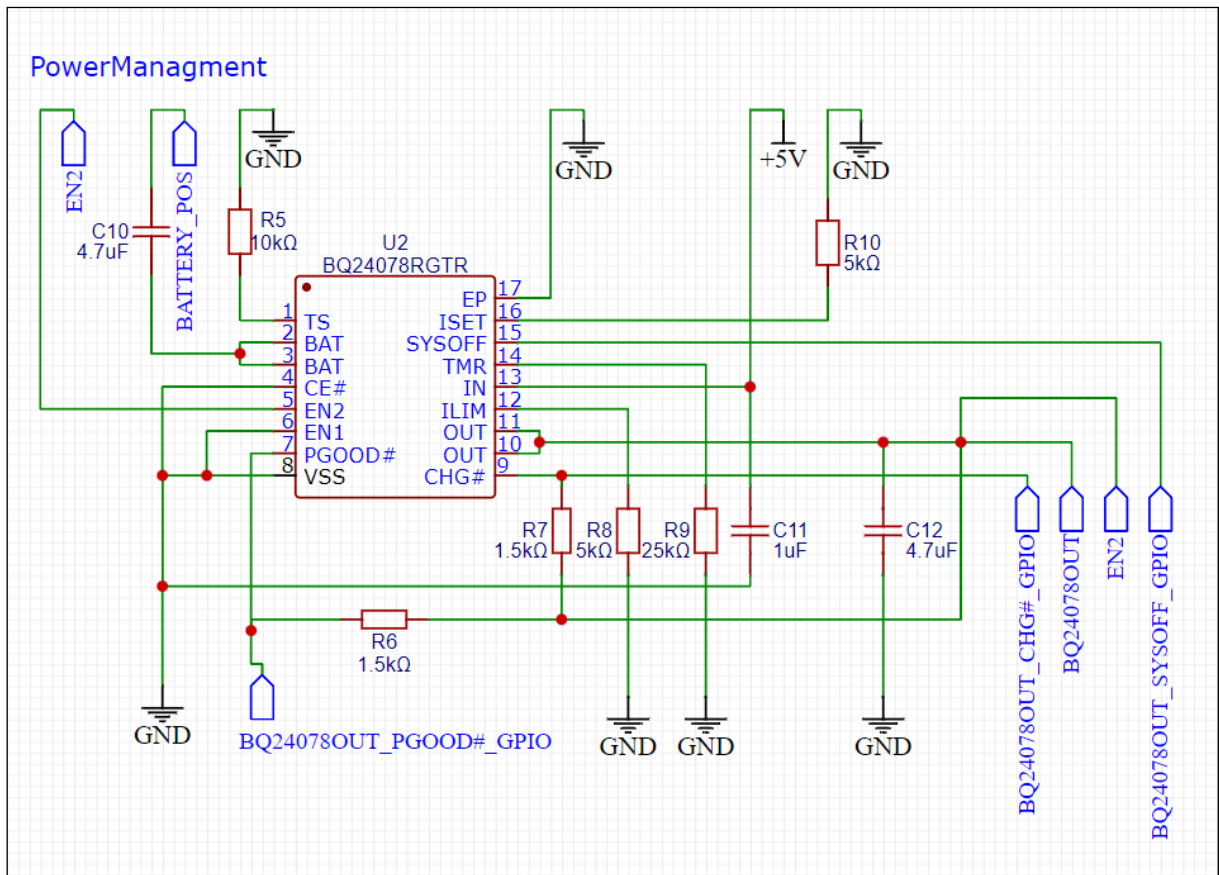
Część zasilania jest kluczowym elementem układu czujnika wilgotności. Zaprojektowana z myślą o energooszczędności i wydajności jaką musi cechować się urządzenie IoT. Zapewnia stabilne zasilanie najważniejszych elementów urządzenia. Składa się z układu zarządzania ładowaniem baterii i rozprowadzania mocy, stabilizatora napięcia oraz interfejsów zewnętrznych takich jak złącze USB-C czy gniazdo baterii.



Rysunek 4 Szczegółowy schemat pierwszej sekcji części zasilania

W projekcie do ładowania urządzenia zastosowano złącze USB-C. Obsługuje napięcie wejściowe równe 5 V i zabezpiecza układ przed nadmiernym napięciem oraz prądem. Aby zredukować szумы i zakłócenia, napięcie wejściowe jest filtrowane przez kondensatory  $C_1$  i  $C_2$  o pojemnościach 100 nF, co dodatkowo stabilizuje napięcie zasilania. Piny VBUS zostały połączone z wejściem układu bq24078, umożliwiając ładowanie baterii oraz zasilanie pozostałych komponentów. Piny CC<sub>1</sub> i CC<sub>2</sub> wykorzystywane są do wykrywania połączenia kabla USB-C, przy czym zastosowane rezystory ściągające R<sub>1</sub> i R<sub>2</sub> o wartościach 5,1 kΩ pozwalają na poprawną identyfikację podłączonego złącza.





Rysunek 5 Szczegółowy schemat drugiej sekcji części zasilania

Układ bq24078 odpowiada za zarządzanie procesem ładowania baterii litowo-jonowej oraz dystrybucję energii w urządzeniu. Projekt został opracowany zgodnie z wytycznymi producenta [18], zapewniając stabilność i bezpieczeństwo pracy systemu. Do stabilizacji napięcia w układzie zastosowano kondensatory C<sub>10</sub>, C<sub>11</sub> i C<sub>12</sub> o wartościach odpowiednio 4,7 µF, 1 µF oraz 4,7 µF. Komponenty te zostały dobrane zgodnie z zaleceniami producenta, co gwarantuje odpowiednią filtrację i stabilizację napięcia w układzie. Pin BAT został połączony z dodatnim biegunem baterii, natomiast pin TS, zwykle stosowany do monitorowania temperatury za pomocą termistora, został w tym projekcie podłączony do masy poprzez rezystor R<sub>5</sub> o wartości 10 kΩ, ponieważ funkcja monitorowania temperatury nie jest wykorzystywana. Wejścia logiczne EN<sub>1</sub> i EN<sub>2</sub> umożliwiają konfigurację limitu prądu ładowania. W projekcie ustawiono kombinację stanów logicznych 0 i 1, co zgodnie z dokumentacją producenta [18] pozwala na limit prądu wejściowego wynoszący 1 A. Dodatkowo, precyzyjna wartość limitu prądu została ustalona za pomocą rezystora R<sub>8</sub> o wartości 5kΩ, zgodnie ze wzorem:

$$I_{input\ limit} = \frac{V_{ILIM}}{R_{ILIM} * K_{ILIM}},$$

gdzie V<sub>ILIM</sub> wynosi 1,5 V, a K<sub>ILIM</sub> wynosi 1300. Przyjęta wartość rezystora R<sub>8</sub> ustala maksymalny prąd wejściowy na około 230 mA.

Prąd ładowania baterii został określony za pomocą rezystora ISET. Wartość rezystora  $R_{10}$ , wynosząca 5 k $\Omega$ , została dobrana na podstawie wzoru:

$$I_{charge} = \frac{K_{ISET}}{R_{ISET}},$$

gdzie  $K_{ISET} = 1000$ . Ostatecznie prąd ładowania wynosi 200 mA, co zwiększa czas ładowania urządzenia, ale jednocześnie zapewnia wysokie bezpieczeństwo i wydłużoną żywotność baterii. Pin  $\overline{CE}$  odpowiedzialny za włączanie i wyłączanie funkcji ładowania, został podłączony do masy, co pozwala na stałą gotowość do ładowania bez potrzeby dodatkowego sterowania. Opcjonalny pin SYSOFF umożliwia zasilanie układu podczas ładowania baterii, natomiast wyjście  $\overline{CHG}$ , podłączone do GPIO mikrokontrolera, monitoruje stan ładowania. Pin  $\overline{PGOOD}$  sygnalizuje obecność zewnętrznego źródła zasilania. Rezystory podciągające  $R_6$  i  $R_7$  o wartościach 1,5 k $\Omega$  zostały dobrane zgodnie z kartą katalogową układu. Pin TMR odpowiada za kontrolę maksymalnego czasu ładowania, który w projekcie został ustawiony na 6 godzin i 15 minut. Wartość rezystora  $R_9$  (25 k $\Omega$ ) została dobrana na podstawie wzoru:

$$R_{TMR} = \frac{t_{timer}}{K_{TMR}},$$

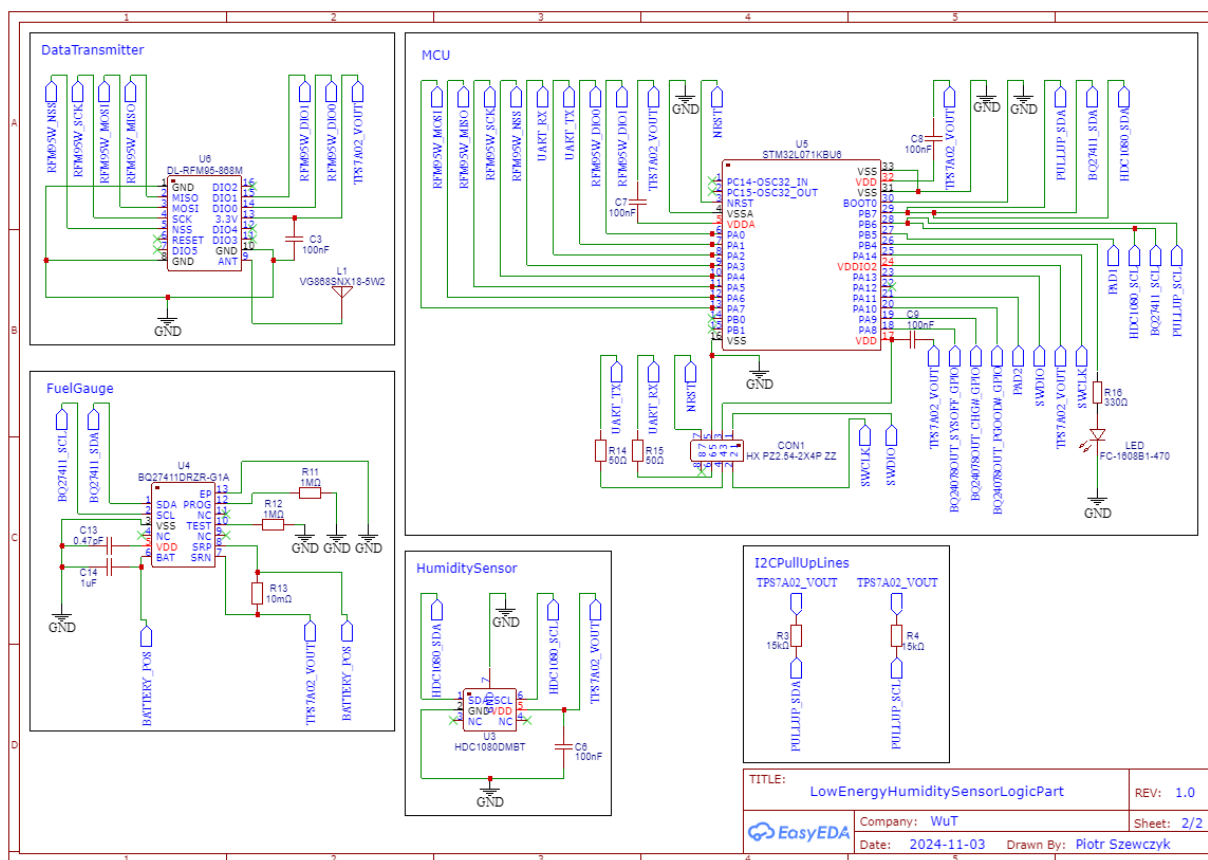
gdzie  $K_{TMR}$  wynosi  $0,25 \frac{\text{godzina}}{\text{k}\Omega}$ , co pozwala na automatyczne wyłączenie ładowania po tym czasie. Przyjęty czas chroni urządzenie przed przegrzaniem i uszkodzeniem. Ze względu na dłuższy czas ładowania (ok. 12 godzin), użytkownik będzie musiał podłączyć urządzenie dwukrotnie, co jest kompromisem pomiędzy wygodą a bezpieczeństwem i trwałością urządzenia. Czas potrzebny do pełnego naładowania baterii został oszacowany na podstawie pojemności baterii  $C_{baterii} = 2000$  mAh i prądu ładowania  $I_{\text{ładowania}} = 200$  mA:

$$t_{\text{ładowania}} = \frac{C_{baterii}}{I_{\text{ładowania}}} = \frac{2000\text{mAh}}{200\text{mA}}$$

$$t_{\text{ładowania}} = 10 \text{ godzin}$$

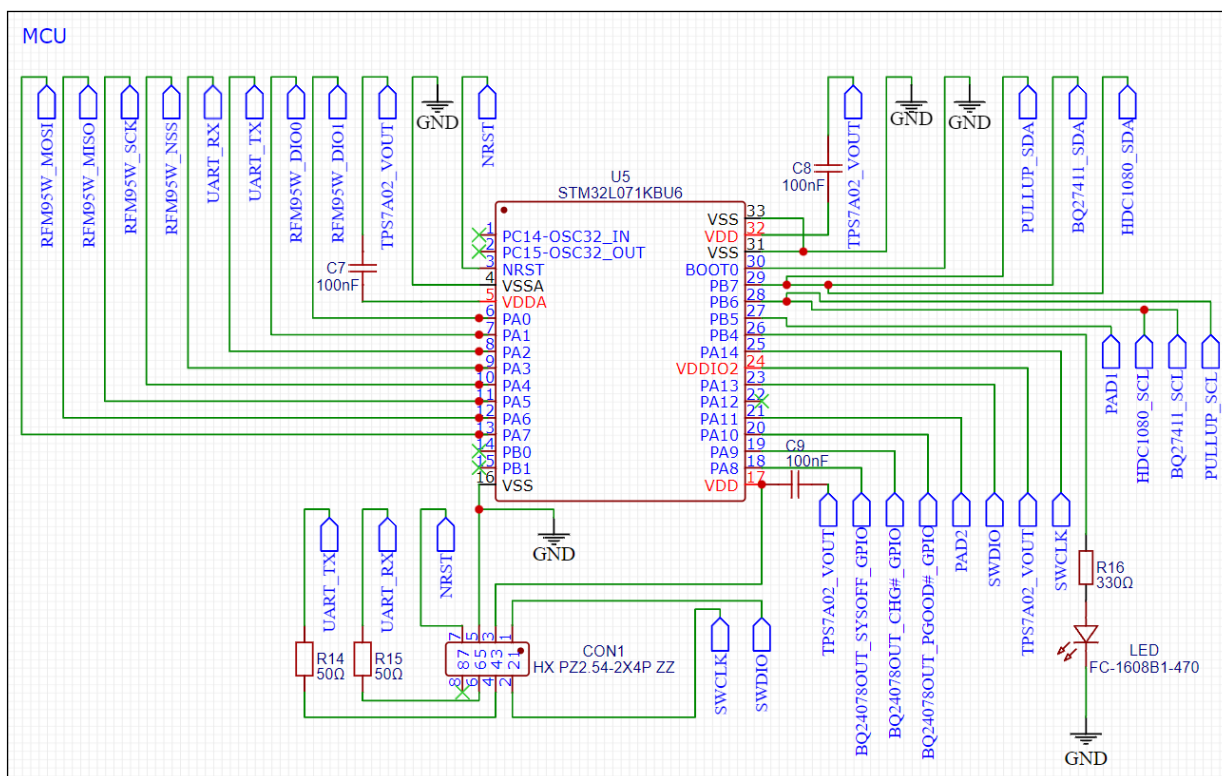
Projekt układu z bq24078 został zoptymalizowany pod kątem bezpieczeństwa i trwałości urządzenia, przy zachowaniu odpowiedniej funkcjonalności. Pomimo wydłużonego czasu ładowania, zastosowane komponenty i rozwiązania zapewniają stabilność pracy, niezawodność oraz niską częstotliwość konieczności ładowania urządzenia dzięki energooszczędnemu charakterowi systemu. Takie ustawienia miały przede wszystkim pozwolić na sprawdzenie logiki układu, prewencyjnie chroniąc go przed zbyt dużym natężeniem prądu.

### 3.3.3 Część cyfrowa



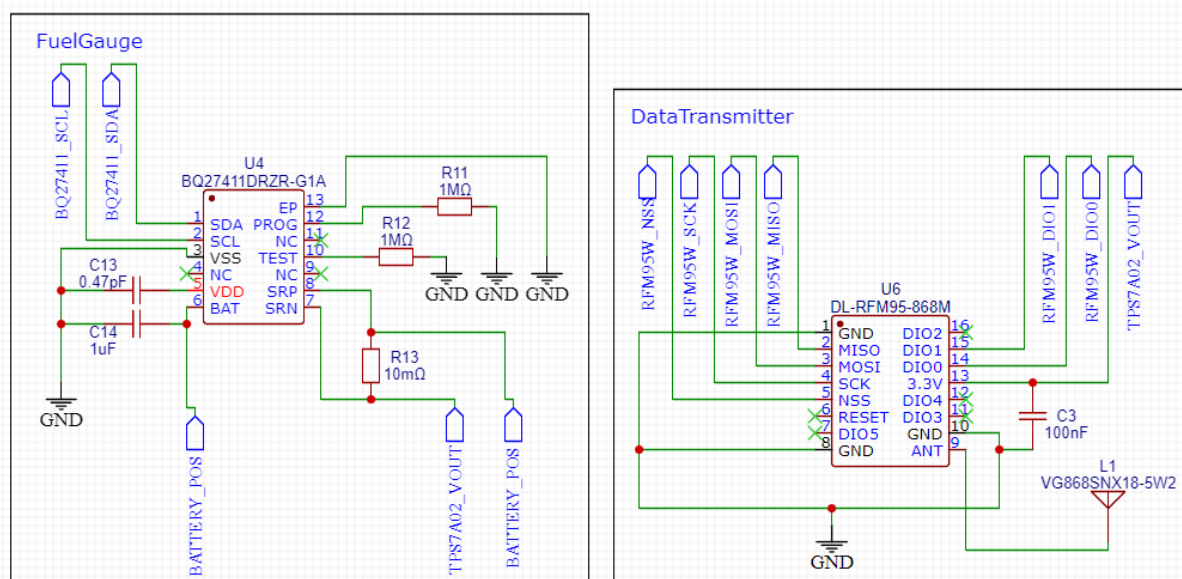
### Rysunek 6 Część cyfrowa

Część cyfrowa odpowiada za przetwarzanie danych, zarządzanie komunikacją z innymi komponentami systemu oraz przesyłanie informacji za pomocą nadajnika LoRa. Sercem układu jest mikrokontroler STM32 z serii L0, odpowiadający za komunikację między elementami (I<sup>2</sup>C, SPI, GPIO) jak i ustawienia energooszczędności.



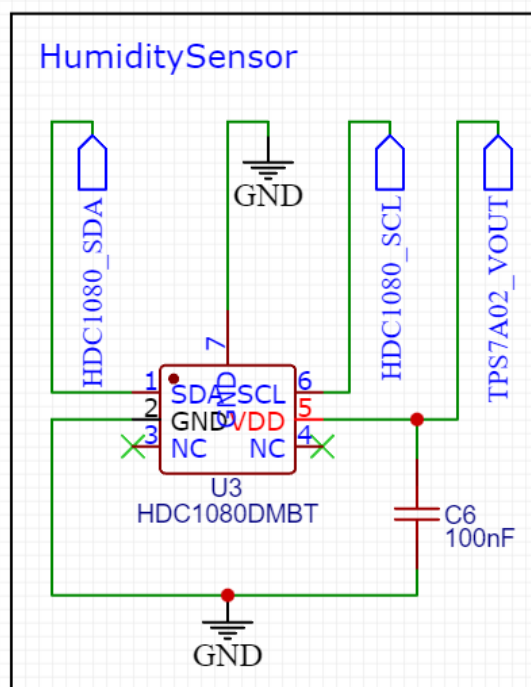
Rysunek 7 Szczegółowy schemat pierwszej sekcji części cyfrowej

W projekcie mikrokontroler STM32L071KBU6 został precyzyjnie skonfigurowany, a jego piny pełnią określone funkcje, dostosowane do wymagań systemu. Piny PC14 i PC15, które standardowo obsługują zewnętrzny oscylator krystaliczny, pozostają niepodłączone, ponieważ projekt nie przewiduje jego zastosowania. To prostsze rozwiązanie redukuje liczbę elementów na płycie, jednocześnie pozwalając na stabilną pracę mikrokontrolera dzięki wbudowanemu oscylatorowi. Sekcja komunikacji mikrokontrolera z komputerem została zrealizowana za pomocą interfejsu UART. Piny NRST, PA14 (SWCLK), PA13 (SWDIO), PA2 (UART\_TX) oraz PA3 (UART\_RX) zostały wyprowadzone na ośmio-pinowe gniazdo typu goldpin, umożliwiając łatwy dostęp do programowania i diagnostyki. Rezystory R<sub>14</sub> i R<sub>15</sub>, o wartościach 50 Ω każdy, podciągają linie UART, co zapewnia stabilność sygnałów. Komunikacja z kluczowymi komponentami systemu, takimi jak nadajnik LoRa RFM95, czujnik wilgotności HDC1080 oraz miernik stanu baterii bq27411, została starannie zaplanowana. Piny PA4, PA5, PA6, PA7 odpowiadają za komunikację SPI z modulem RFM95, a dodatkowe piny PA0 i PA1 wspierają jego funkcje. Z kolei linia I<sup>2</sup>C, obsługująca HDC1080 i bq27411, wykorzystuje piny PB6 i PB7, a dwa rezystory podciągające R<sub>3</sub> i R<sub>4</sub>, o wartościach 15 kΩ, zapewniają odpowiednią pracę magistrali. Niektóre piny mikrokontrolera zostały przeznaczone na dodatkowe funkcjonalności. Piny PB5 i PA11 są zarezerwowane jako uniwersalne wyprowadzenia GPIO, umożliwiając rozbudowę systemu w przyszłości. Pin PB4 został połączony z programowalną diodą LED, która może sygnalizować różne stany systemu, na przykład podłączenie baterii lub trwające ładowanie. Aby zapewnić stabilne zasilanie mikrokontrolera, zastosowano kondensatory filtrujące C<sub>7</sub>, C<sub>8</sub> i C<sub>9</sub>, o pojemnościach 100 nF każdy. Są one rozmieszczone w pobliżu kluczowych punktów zasilania, aby minimalizować szumy i zakłócenia. Całość układu została zaprojektowana tak, aby zapewnić maksymalną funkcjonalność przy jednoczesnym zachowaniu prostoty i niezawodności, umożliwiając łatwą integrację mikrokontrolera z pozostałymi elementami systemu IoT.



Rysunek 8 Szczegółowy schemat drugiej sekcji części cyfrowej

Wyprowadzenia bq27411 TEST i PROG zostały zgodnie z notą katalogową uziemione za pomocą rezystorów  $R_{11}$  i  $R_{12}$  o wartościach  $1\text{ M}\Omega$ , co spełnia wymogi prawidłowej pracy układu. Pin BAT, podłączony do dodatniego bieguna baterii, został wyposażony w kondensator filtrujący  $C_{14}$  o wartości  $1\text{ }\mu\text{F}$ , stabilizujący napięcie na tym połączeniu. Wyjście regulatora wewnętrznego LDO, oznaczone jako VDD, zostało ustabilizowane kondensatorem  $C_{13}$  o wartości  $0,47\text{ pF}$ , co jest zgodne z wymaganiami producenta. Rezystor  $R_{13}$  to opornik pomiarowy. Jego wartość  $10\text{ m}\Omega$  została dobrana dzięki notce katalogowej. Jest umieszczony po stronie wysokiego napięcia, bezpośrednio między dodatnim biegunem akumulatora a wejściem regulatora napięcia TPS7A02, przez którego cały komponent jest też zasilany. Taka konfiguracja zapewnia, że cały prąd płynący z akumulatora – zarówno ten zużywany przez regulator, jak i przez inne komponenty zasilane na szynie  $3,3\text{ V}$  – przechodzi przez  $R_{15}$ . Dzięki temu układ bq27411 może dokładnie mierzyć prąd wypływający z baterii, co jest kluczowe dla precyzyjnego monitorowania zużycia energii. Komunikacja między modułem RFM95 a mikrokontrolerem odbywa się za pomocą interfejsu SPI, z wykorzystaniem pinów MOSI, MISO, SCK oraz NSS, które zostały odpowiednio połączone z mikrokontrolerem, zapewniając płynny transfer danych. Antena o impedancji  $50\text{ }\Omega$ , precyzyjnie dopasowana do wymagań modułu, została podłączona do pinu ANT, co gwarantuje efektywne nadawanie i odbiór sygnałów radiowych. Pin DIO0 pełni funkcję znacznika ukończonej transmisji i może być wykorzystany do synchronizacji działań urządzenia.



Rysunek 9 Szczegółowy schemat trzeciej sekcji części cyfrowej

Między linię zasilania czujnika a masę włączono kondensator  $C_6$  o wartości 100 nF, który stabilizuje napięcie i minimalizuje zakłócenia, zapewniając niezawodną pracę w różnych warunkach. Wyprowadzenia SDA i SCL, obsługujące komunikację przez interfejs I<sup>2</sup>C, zostały podłączone do mikrokontrolera, umożliwiając szybki i efektywny transfer danych. Zasilanie czujnika zapewnia stabilizator napięcia, co gwarantuje jego poprawne działanie w systemie IoT.

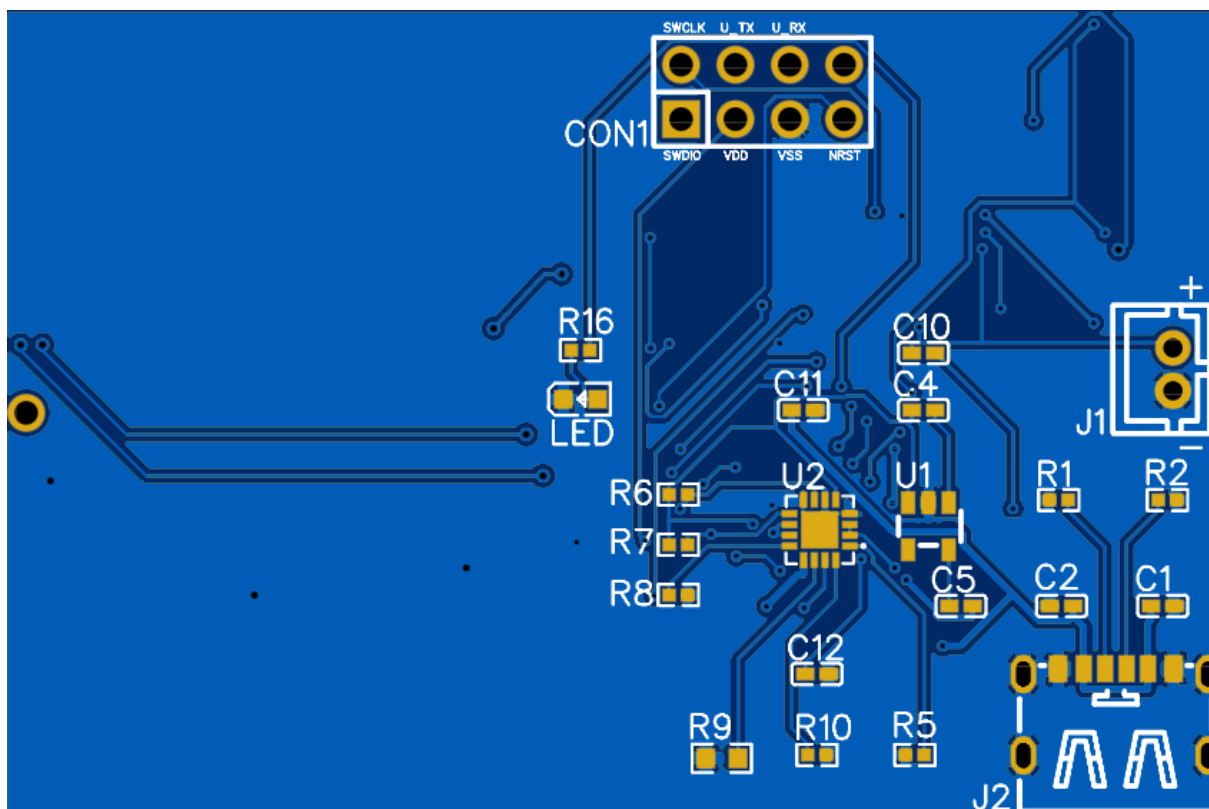
### 3.4 Schemat PCB

Darmowy program EasyEDA w sposób prosty i klarowny umożliwia projektantowi przedstawienie, najpierw założeń połączeń na schemacie elektrycznym, a później intuicyjnie przeniesienie ich na schemat PCB. Pozwala na bezproblemowe wygenerowanie plików Gerber oraz BOM. Sam prototyp oraz montaż elementów wykona firma zewnętrzna – PCBWay [21].

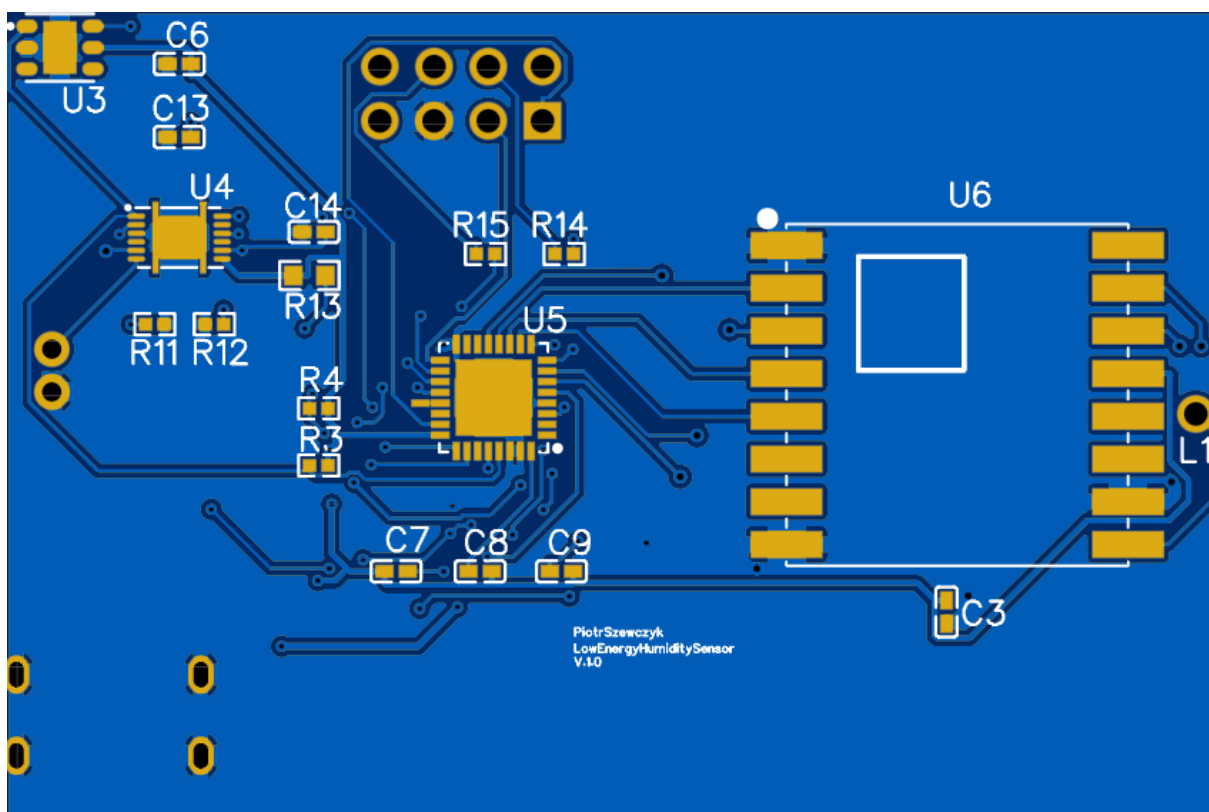
Celem projektu było osiągnięcie jak najmniejszych wymiarów płytki przy zachowaniu pełnej funkcjonalności systemu. Wymiary prototypu zostały określone na 57 mm x 38 mm, co pozwala na idealne dopasowanie do zastosowanej baterii o wymiarach 50 mm x 34 mm. Wysokość całego prototypu wynosi około 12 mm na co składają się grubości płytki (1,6 mm) i baterii (10 mm). Komponenty zostały rozmieszczone w sposób optymalny, co zapewnia krótkie ścieżki sygnałowe oraz minimalizację strat napięcia. Płytką została wykonana na bazie laminatu FR-4 z TG150, który charakteryzuje się wysoką odpornością termiczną.

Jednym z kluczowych założeń projektu było rozdzielenie części zasilania i cyfrowej. Aby zapewnić stabilność elektryczną oraz ograniczyć zakłócenia elektromagnetyczne (EMI), zaprojektowano pełną płaszczyznę masy na obydwu warstwach PCB. Stanowi ona solidne odniesienie dla wszystkich sygnałów, redukując zakłócenia i poprawiając integralność

sygnałów. Kondensatory filtrujące, przypisane danym komponentom zostały rozmieszczone blisko ich pinów zasilających, co zapewnia stabilność napięcia i redukcję szumów. Część zasilania, obejmująca elementy takie jak układ ładowania bq24078, regulator napięcia TPS7A02 oraz kondensatory filtrujące, została zlokalizowana w dolnej części płytki, w pobliżu wejścia zasilania (USB-C i JST). Takie rozmieszczenie pozwala na efektywne prowadzenie zasilania oraz minimalizację zakłóceń przenikających do części cyfrowej. Część cyfrowa, zawierająca mikrokontroler STM32L071KBU6, czujnik wilgotności HDC1080, fuel gauge bq27411 oraz moduł komunikacji LoRa RFM95, została umieszczona na górnej warstwie. W centrum tej warstwy znajduje się mikrokontroler co umożliwia poprowadzenie ścieżek sygnałowych w sposób logiczny, jak najbardziej zoptymalizowany, gdyż jest on głównym komponentem urządzenia. Moduł LoRa oraz czujnik wilgotności zostały umieszczone na dwóch przeciwnych skrajach płytki, co dodatkowo zabezpiecza sygnały przed zakłóceniami. Antena jest poza obszarem masy oraz wystaje poza obrys płytki. Zastosowano odpowiednie parametry ścieżek, przelotek oraz odstępów, zgodnie z obowiązującymi standardami projektowymi. Ścieżki zostały zaprojektowane z trzema różnymi grubościami, dostosowanymi do pełnionych funkcji: dla ścieżek zasilania przyjęto grubość 0,3 mm, co zapewnia stabilne prowadzenie napięcia i minimalizację strat; dla ścieżek regulatora zastosowano grubość 0,25 mm, która łączy odpowiednią przewodność z kompaktowym rozmiarem, natomiast dla ścieżek sygnałowych przyjęto grubość 0,15 mm, wystarczającą dla sygnałów niskonapięciowych i niskoprądowych. Do połączeń między warstwami wykorzystano przelotki o średnicy 0,5 mm, z otworami o średnicy 0,2 mm. Ścieżki wokół przelotek mają szerokość 0,5 mm, co gwarantuje ich odpowiednią wytrzymałość mechaniczną i elektryczną. Minimalny odstęp między ścieżkami i przelotkami wynosi 0,152 mm. Ścieżki starano się poprowadzić jak najkrócej było to tylko możliwe oraz bez rozgałęzień, omijano kąty 90°. Na dwa wolne, nieobsługiwane piny kontrolera nałożono pady, w razie potrzeby rozwinięcia, bądź edycji projektu.



Rysunek 10 Dolna warstwa PCB- część zasilania

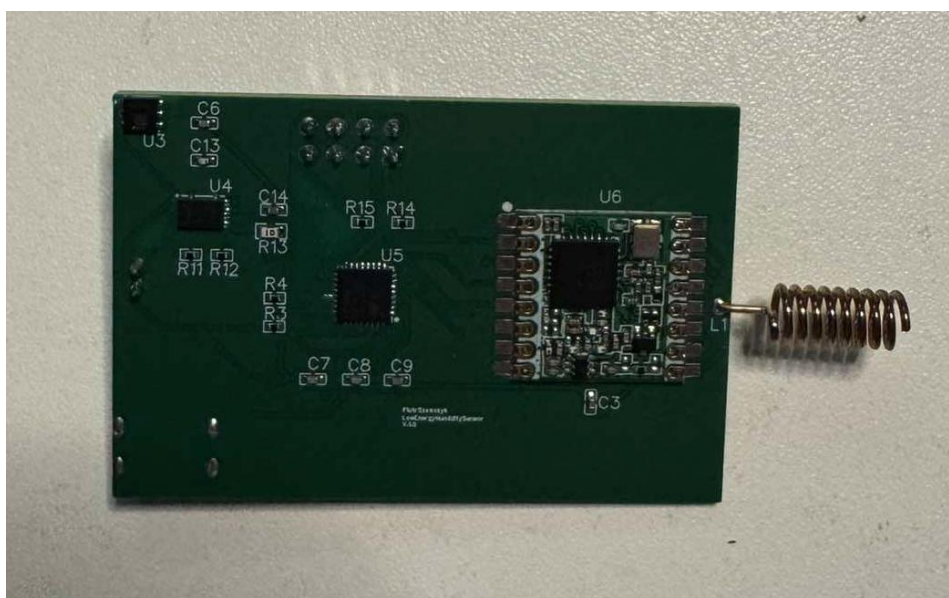


Rysunek 11 Górna warstwa PCB- część cyfrowa





*Rysunek 12 Fotografia prototypu z baterią*



*Rysunek 13 Fotografia prototypu - górna warstwa*

Stworzona płytki PCB została wykonana zgodnie z obowiązującymi standardami projektowania obwodów drukowanych. Starannie przemyślane rozmieszczenie komponentów, ścieżek oraz ich miniaturowe rozmiary sprawiają, że urządzenie idealnie wpasowuje się w założenia projektowe.

## 4 Oprogramowanie, Testowanie, Integracja

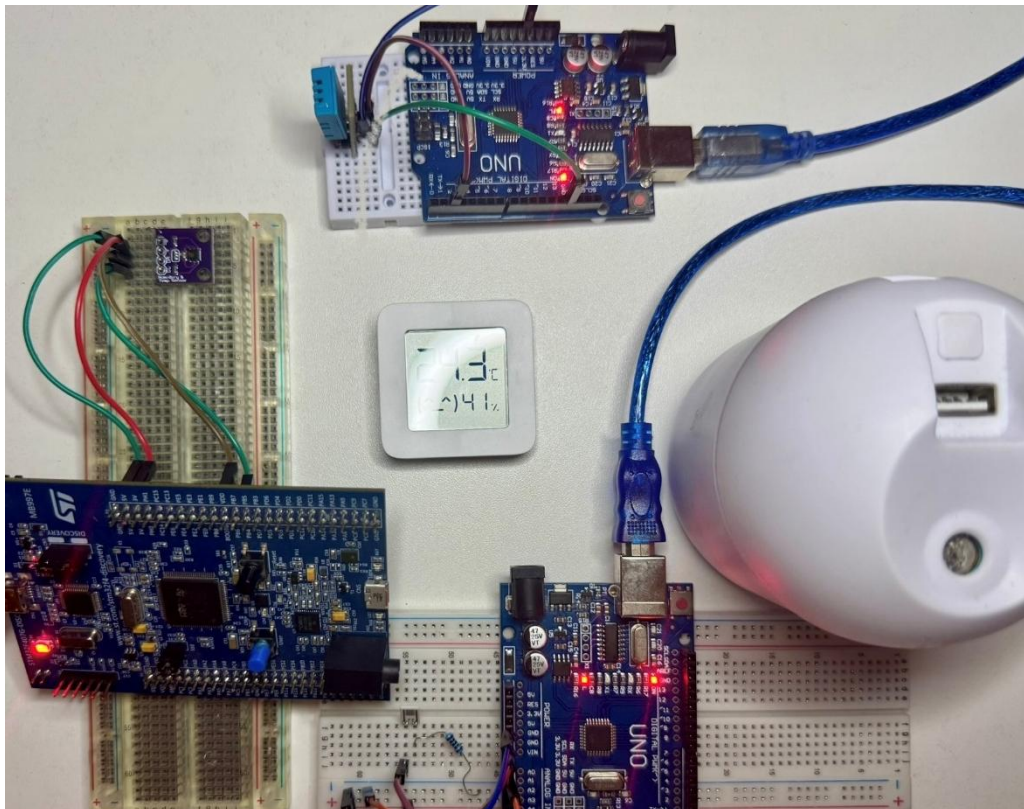
### 4.1 Wprowadzenie

Kolejnym istotnym etapem projektu było opracowanie oprogramowania oraz przeprowadzenie testów zapewniających poprawność działania urządzenia i jego integracji z systemem Home Assistant. Proces obejmował kilka kluczowych etapów: testowanie czujników i komunikacji, implementację transmisji danych za pomocą modułu LoRa oraz weryfikację energooszczędności urządzenia. Każdy z tych kroków został szczegółowo opisany, a wyniki zaprezentowano w formie zdjęć, wyników oraz fragmentów kodu. Samo oprogramowanie zostało napisane w dostępnym za darmo środowisku programistycznym STM32CubeIDE, które oferuje szereg zalet, takich jak zintegrowany edytor kodu oparty na platformie Eclipse, zapewniający wsparcie dla zaawansowanych funkcji edycji i nawigacji w kodzie. Dodatkowo, dzięki integracji z STM32CubeMX, umożliwia automatyczną konfigurację peryferiów mikrokontrolera, co pozwala na szybkie generowanie odpowiedniego kodu. Narzędzia debuggera w STM32CubeIDE oferują debugowanie w czasie rzeczywistym, pozwalając na monitorowanie pracy mikrokontrolera oraz szybkie wykrywanie błędów w kodzie. Środowisko to obsługuje całą rodzinę mikrokontrolerów STM32, co ułatwia przenoszenie kodu między projektami. W trakcie prac wykorzystano również funkcjonalność semihostingu, która pozwala na komunikację między mikrokontrolerem a komputerem za pomocą debuggera. Semihosting umożliwia np. wyświetlanie komunikatów debugowania lub logów w konsoli komputera, co znacznie usprawnia proces testowania i analizy działania aplikacji.

## 4.2 Testy modułarne

Na początkowym etapie realizacji projektu przeprowadzono szczegółowe testy czujników wilgotności na płytkach rozwojowych STM32F407G-DISC1 [22] oraz zamiennikach Arduino Uno [23]. Płytki te umożliwiają szybkie prototypowanie oraz testowanie oprogramowania bez konieczności posiadania własnego układu, które w danym momencie wykonywała firma zewnętrzna.

### 4.2.1 Testy czujników wilgotności



*Rysunek 14 Stanowisko pomiarowe czujników*

Wykorzystano moduł cyfrowego czujnika HDC1080, który został zintegrowany z STM32F407G-DISC1 za pomocą interfejsu I<sup>2</sup>C. Przygotowano kod testowy, pozwalający na odczyt danych z czujnika oraz ich analizę.

```

#include "hdc1080testing.h"

#define HDC1080_TEMPERATURE_REG 0x00
#define HDC1080_HUMIDITY_REG 0x01

HAL_StatusTypeDef HDC1080_Init(I2C_HandleTypeDef *hi2c) {
    uint8_t config[3] = {0x02, 0x10, 0x00}; // default config
    return HAL_I2C_Master_Transmit(hi2c, HDC1080_ADDR, config, 3, HAL_MAX_DELAY);
}

HAL_StatusTypeDef HDC1080_Read(I2C_HandleTypeDef *hi2c, HDC1080_Data *data) {
    uint8_t reg = HDC1080_TEMPERATURE_REG;
    uint8_t buffer[4];

    // Trigger t&h measure
    if (HAL_I2C_Master_Transmit(hi2c, HDC1080_ADDR, &reg, 1, HAL_MAX_DELAY) != HAL_OK)
        return HAL_ERROR;

    HAL_Delay(20); // Measure

    if (HAL_I2C_Master_Receive(hi2c, HDC1080_ADDR, buffer, 4, HAL_MAX_DELAY) != HAL_OK)
        return HAL_ERROR;

    // raw->real
    uint16_t raw_temp = (buffer[0] << 8) | buffer[1];
    uint16_t raw_humidity = (buffer[2] << 8) | buffer[3];

    data->temperature = (raw_temp / 65536.0) * 165.0 - 40.0; // Convert to °C
    data->humidity = (raw_humidity / 65536.0) * 100.0; // Convert to %

    return HAL_OK;
}

```

Rysunek 15 Fragment kodu źródłowego *hdc1080testing.c*

Ten fragment kodu przedstawia funkcje obsługujące czujnik HDC1080, które realizują konfigurację urządzenia oraz odczyt pomiarów temperatury i wilgotności za pomocą interfejsu I<sup>2</sup>C. Funkcja *HDC1080\_Init* odpowiada za inicjalizację czujnika, wysyłając domyślne ustawienia w postaci tablicy *config*, co pozwala na jego poprawne działanie. Funkcja *HDC1080\_Read* wykonuje odczyt danych pomiarowych, rozpoczynając od wysłania komendy pomiaru temperatury i wilgotności. Następnie wprowadza opóźnienie (*HAL\_Delay*), aby umożliwić czujnikowi przeprowadzenie pomiaru. Po zakończeniu pomiaru funkcja odbiera dane z czujnika w postaci czterech bajtów, gdzie dwa pierwsze reprezentują temperaturę, a kolejne dwa wilgotność. Otrzymane surowe wartości są przeliczane na rzeczywiste wartości temperatury (°C) i wilgotności (%) zgodnie ze wzorami dostarczonymi przez producenta, a wyniki zapisywane są w strukturze *HDC1080\_Data*. Kod wykorzystuje funkcje z biblioteki HAL, realizując standardowe operacje transmisji i odbioru danych w interfejsie I<sup>2</sup>C.

Czujniki DHT11 (jeden z najpopularniejszych cyfrowych czujników wilgotności [24]) oraz analogowy SYH-2R [25] zostały przetestowane na płytkach Arduino Uno. Kody dla Arduino Uno dołączone zostały w załącznikach pracy.

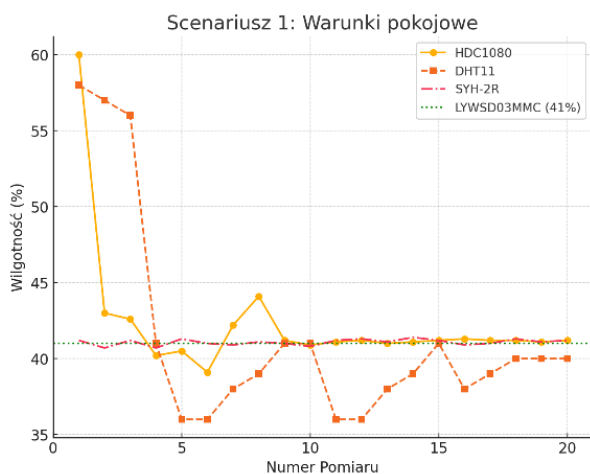
## Wyniki Testów:

Na pierwszym etapie testy zostały przeprowadzone w standardowych warunkach pokojowych, gdzie higrometr referencyjny (Mi Temperature and Humidity Monitor 2 [26]) wskazywał 41% wilgotności. Wykazano, że analogowy czujnik SYH-2R od razu dostosowywał się do warunków i dawał wyniki zgodne z wartością referencyjną. Z kolei czujniki cyfrowe (HDC1080 i DHT11) wymagały czasu na stabilizację pomiarów. Pierwsze wartości, były zawyżone (około 60%), zanim ustabilizowały się do oczekiwanych wyników w pobliżu wartości referencyjnej.

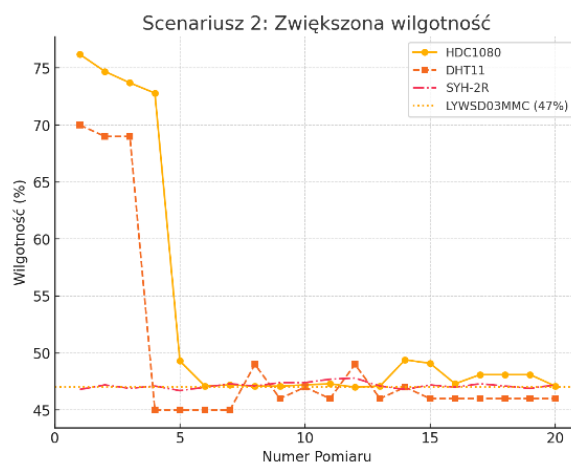
W drugim etapie wokół stanowiska włączono nawilżacz powietrza i pozostawiono go włączonym na 4 godziny, co podniosło wilgotność w pomieszczeniu do 47%. Rezultaty były identyczne jak w pierwszym scenariuszu.

Na trzecim etapie stanowisko testowe zostało przeniesione do strefy o bardzo wysokiej wilgotności, gdzie higrometr wskazał 82%. Strefę tę utworzono poprzez rozlanie wody oraz zastosowanie zamkniętego pojemnika zwiększającego wilgotność powietrza. Wyniki potwierdziły, że analogowy czujnik SYH-2R bardzo szybko reagował na zmiany, jednak czasem wskazywał bardzo odbiegające od rzeczywistości wartości. HDC1080 wciąż był bardzo bliski referencji, natomiast DHT11 delikatnie odbiegał z precyzją

W finalnym projekcie zdecydowano się na zastosowanie czujnika HDC1080. Ze względu na konieczność stabilizacji pomiarów, pierwsze 5 odczytów jest pomijane, a średnia z kolejnych 5 odczytów służy jako końcowy wynik pomiarowy. Analizując pozostałe czujniki, zdecydowano o rezygnacji z zastosowania SYH-2R, jako reprezentanta rodziny czujników analogowych, ze względu na podatność na zakłócenia oraz brak odporności na szумы wraz ze wzrostem skomplikowania układu i jego miniaturyzacją. Dodatkowo taki czujnik każdorazowo trzeba odpowiednio skalibrować do odpowiadających warunków co wyklucza go z projektu. Czujnik taki jak DHT11 ustępował precyzją HDC1080 oraz zużyciem energii, co również wpłynęło na odrzucenie takowego rozwiązania.

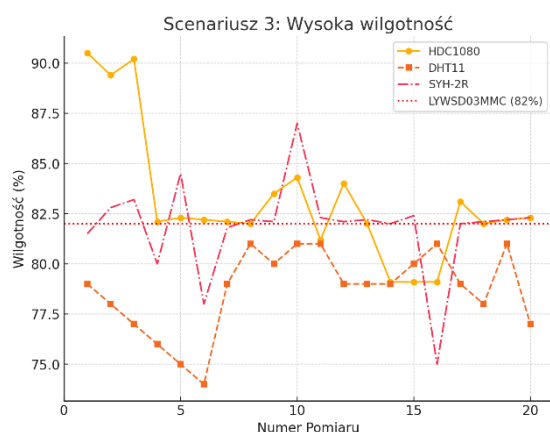


Rysunek 16 Wyniki I scenariusza testowego



Rysunek 17 Wyniki II scenariusza testowego





Rysunek 18 Wyniki III scenariusza testowego

Przeprowadzone testy dowiodły, że wybór HDC1080 jest optymalny pod względem precyzji, stabilności, elastyczności oraz łatwości integracji z resztą systemu. Wyniki te posłużyły jako podstawa do implementacji finalnego rozwiązania wczesnego wykrywania wilgotności.

#### 4.2.2 Testowanie LoRa

W kolejnym etapie skupiono się na integracji danych z czujnika HDC1080 z systemem IoT za pomocą modułów LoRa RA-01 SX1278 433MHz [13]. Moduły te są bardzo zbliżone do RFM95, co ułatwia przenoszenie kodu i implementację. Jeden z modułów SX1278 został skonfigurowany jako nadajnik i podłączony do płytki STM32F407G-DISC1. Moduł ten odpowiadał za wysyłanie danych pomiarowych z czujnika HDC1080. Drugi moduł LoRa został skonfigurowany jako odbiornik i podłączony do Raspberry Pi 4 [27]. Obydwa moduły komunikują się ze swoją jednostką sterującą za pomocą interfejsu komunikacyjnego SPI. Na komputerze uruchomiono wirtualną maszynę z zainstalowanym systemem Home Assistant. Odbiornik dostawał dane przesyłane przez nadajnik i przysyłał je do Home Assistant, gdzie mogły być wizualizowane oraz wykorzystane do tworzenia automatyzacji. Takie podejście pozwala skutecznie zintegrować urządzenie z Internetem Rzeczy, zapewniając stabilną i efektywną transmisję danych.

```

    // Set frequency to 433 MHz
    LoRa_SetFrequency(FRF_MSB, FRF_MID, FRF_LSB);

    // Configure Modem: BW=125kHz, CR=4/5, SF=7, CRC ON
    write_register(REG_MODEM_CONFIG1, 0x72); // BW=125kHz, CR=4/5
    write_register(REG_MODEM_CONFIG2, 0x74); // SF=7, CRC ON

    //Set to Standby Mode
    write_register(REG_OP_MODE, MODE_STDBY);
}

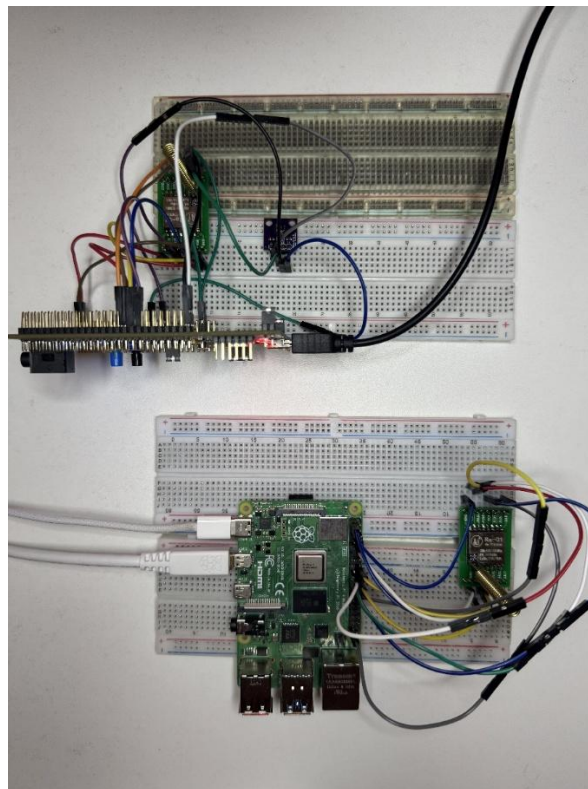
// Set LoRa Frequency
void LoRa_SetFrequency(uint8_t msb, uint8_t mid, uint8_t lsb) {
    write_register(REG_FRF_MSB, msb);
    write_register(REG_FRF_MID, mid);
    write_register(REG_FRF_LSB, lsb);
    .
    .
    .
// Send a Message
void LoRa_SendMessage(const char *message) {
    uint8_t length = strlen(message);
    // Set to Standby Mode
    write_register(REG_OP_MODE, MODE_STDBY);
    // Set FIFO pointer
    write_register(REG_FIFO_ADDR_PTR, 0x00);
    // Load data into FIFO
    for (uint8_t i = 0; i < length; i++) {
        write_register(REG_FIFO, message[i]);
    }
    // Set payload length
    write_register(REG_PAYLOAD_LENGTH, length);
    // Switch to Transmit Mode
    write_register(REG_OP_MODE, MODE_TX);
    // Wait for TX Done with a timeout
    uint32_t timeout = HAL_GetTick() + 2000; // 2-second timeout
    while ((read_register(REG_IRQ_FLAGS) & 0x08) == 0) {
        if (HAL_GetTick() > timeout) {
            printf("Error: TX timeout\n");
            break;
        }
    }
    // ClrIRQflags
    write_register(REG_IRQ_FLAGS, 0xFF);

    // Standby
    write_register(REG_OP_MODE, MODE_STDBY);
}

```

Rysunek 19 Fragment kodu źródłowego loratesting.c

Kod realizuje obsługę modułu LoRa, obejmując kluczowe funkcjonalności: ustawianie częstotliwości oraz wysyłanie wiadomości. Proces rozpoczyna się od inicjalizacji modułu, realizowanej przez funkcję *LoRa\_Init*. Funkcja ta resetuje moduł za pomocą manipulacji pinem GPIO, a następnie ustawia jego częstotliwość na 433 MHz oraz konfiguruje jego parametry, takie jak szerokość pasma (125 kHz), kodowanie (4/5) i włączenie CRC. Wszystkie ustawienia są realizowane poprzez zapis odpowiednich wartości do rejestrów przy użyciu interfejsu SPI. Ustawianie częstotliwości odbywa się za pomocą funkcji *LoRa\_SetFrequency*, która zapisuje wartości częstotliwości do rejestrów *REG\_FRF\_MSB*, *REG\_FRF\_MID* i *REG\_FRF\_LSB*. Takie podejście umożliwia precyzyjne określenie parametrów pracy modułu. Ostatnią kluczową funkcjonalnością jest wysyłanie wiadomości, realizowane przez funkcję *LoRa\_SendMessage*. Dane są ładowane do bufora FIFO, a następnie ustawiana jest ich długość i moduł przełączany zostaje w tryb nadawania. Funkcja monitoruje zakończenie transmisji, kontrolując *timeout*, a po zakończeniu czyszczone są flagi IRQ, co pozwala na kolejne operacje.

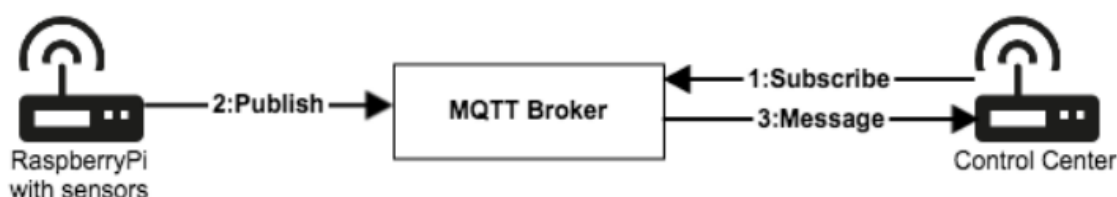


Rysunek 20 Stanowisko testowe LoRa



## 4.3 Integracja z Home Assistant

Home Assistant to otwarte, darmowe oprogramowanie do automatyzacji domu, która umożliwia integrację inteligentnych urządzeń i tworzenie zaawansowanych reguł automatyzacji. System ten jest szczególnie przydatny w projektach IoT, w których zbierane dane z różnych czujników mogą być przetwarzane i wykorzystywane do automatyzacji działań, takich jak informowanie użytkownika o wydarzeniach, sterowanie urządzeniami czy analiza danych w czasie rzeczywistym. Do integracji systemu Home Assistant z odbiornikiem podłączonym do Raspberry Pi 4, potrzebne jest użycie protokołu komunikacyjnego, który działać będzie jako pośrednik przekazywania danych. MQTT (ang. Message Queuing Telemetry Transport) to lekki i prosty w obsłudze protokół komunikacyjny, idealny do przesyłania danych w systemach IoT [28], szczególnie w urządzeniach o ograniczonych zasobach, takich jak te korzystające z technologii LoRa.



Rysunek 21 Schemat komunikacji MQTT [28]

W projekcie centralną rolę komunikacji odgrywa Raspberry Pi 4, które skonfigurowane zostało jako broker MQTT, korzystając z oprogramowania Mosquitto MQTT. Raspberry Pi 4 wyposażone w moduł LoRa RA-01 odbiera sygnały bezprzewodowe przesyłane przez nadajnik. Skrypt (`lora_receiver.py`) uruchomiony na Raspberry Pi dekoduje te dane i publikuje je w odpowiednich tematach do brokera MQTT.

```
# data->MQTT
def publish_to_mqtt(temperature, humidity):
    client = mqtt.Client("LoRa_Receiver")
    client.connect("localhost", 1883, 60)
    client.loop_start()

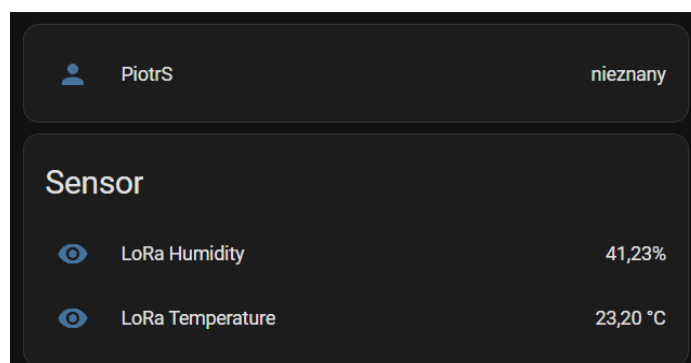
    if temperature is not None and humidity is not None:
        client.publish("home/lora/temperature", temperature)
        client.publish("home/lora/humidity", humidity)
```

Rysunek 22 Fragment kodu źródłowego `lora_receiver.py`

W pliku `lora_receiver.py` poza skonfigurowaną subskrypcją, umieszczona została również konfiguracja odbiornika obejmująca jego parametry transferu danych oraz komunikacja SPI. Dzięki takiemu rozwiązaniu Home Assistant może nasłuchiwać tematów takich jak `home/lora/temperature` i `home/lora/humidity`, aby odbierać dane w czasie rzeczywistym. Ten w swoim pliku konfiguracyjnym `.yaml` (język używany przez Home Assistant) ma dodane wpisy określające, jakie tematy MQTT są subskrybowane.

```
mqtt:
  sensor:
    name: "Lora Temperature"
    state_topic: "home/lora/temperature"
    unit_of_measurement: "°C"
    name: "Lora Humidity"
    state_topic: "home/lora/humidity"
    unit_of_measurement: "%"
```

Rysunek 23 Kod źródłowy configuration.yaml (nasłuch Home Assistant)

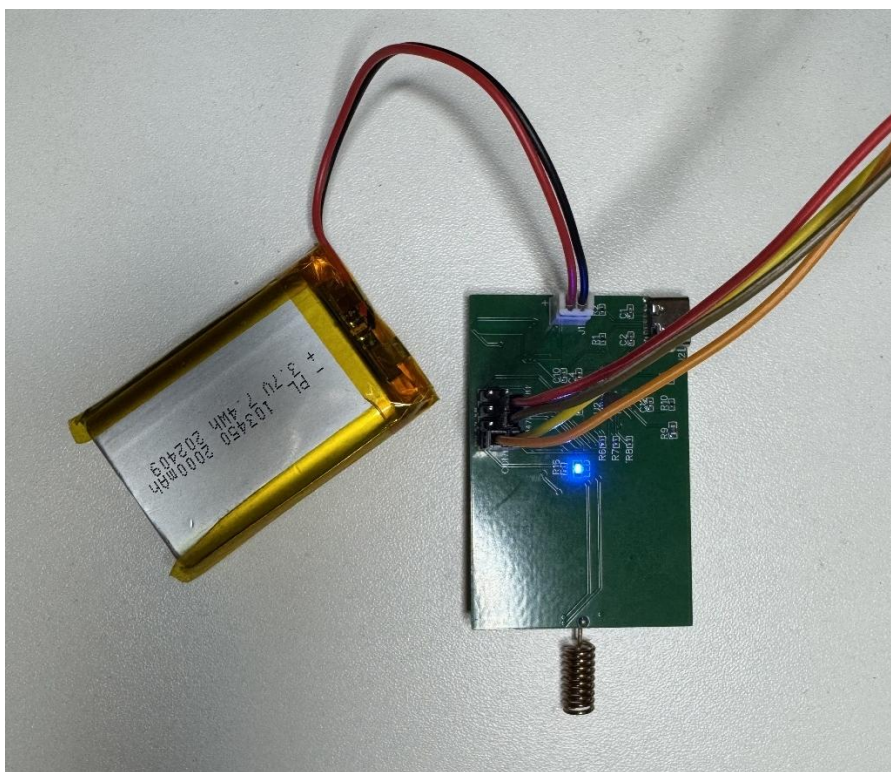


Rysunek 24 Fragment panelu głównego Home Assistant z danymi

#### Wyniki testów transmisji danych:

Podczas testów systemu transmisji LoRa sprawdzono niezawodność przesyłu danych w różnych warunkach. Na początku nadajnik umieszczono blisko odbiornika (1-2 metry), gdzie dane były przesyłane bezproblemowo. Następnie oddalano nadajnik w otwartej przestrzeni, osiągając odległość około 10 metrów, przy zachowaniu stabilności transmisji. W kolejnych testach umieszczono nadajnik za ścianą, w skrytce na liczniki gazu oraz na klatce schodowej w kamienicy. Mimo obecności przeszkód sygnał pozostawał stabilny, a dane poprawnie docierały do odbiornika i były przekazywane do Home Assistant za pomocą MQTT. Testy potwierdziły niezawodność systemu w różnych scenariuszach, zarówno w standardowej terenie, jak i w środowisku z przeszkodami. co pozwala na przypuszczenie poprawnego działania właściwego urządzenia.

## 4.4 Prototyp



Rysunek 25 Fotografia prototypu - dolna warstwa wraz z debuggerem ST-LINK V2

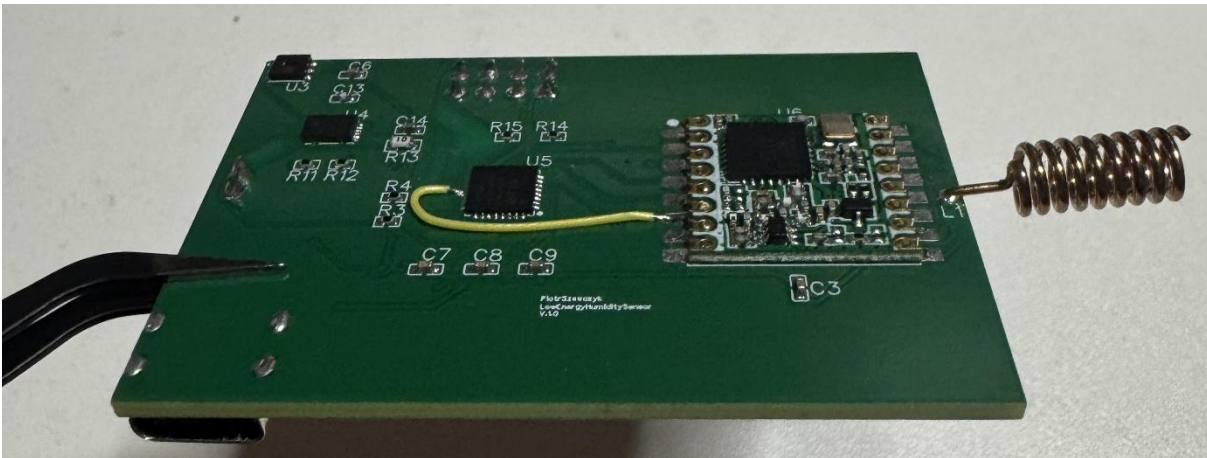
Wraz z przejściem z testów modularnych na dostarczony, wyprodukowany prototyp do oprogramowania dodano kolejne potrzebne funkcjonalności, by spełnić oczekiwania projektu. Kolejnym etapem procesu było testowanie całego urządzenia i jego wszystkich parametrów. Do debuggowania oprogramowania użyto ST-LINK V2, specjalnego narzędzia programistycznego dla rodziny procesorów STM32 oraz STM8.

### 4.4.1 Analiza problemów napotkanych podczas realizacji prototypu

Pierwszym istotnym problemem jest niedziałające ładowanie baterii prototypu za pośrednictwem bq24078 oraz złącza USB-C. Choć projekt zakładał, że bateria będzie ładowana napięciem 5 V, a układ będzie zasilany z tego samego źródła, nie udało się osiągnąć poprawnej pracy tej części systemu. Analiza problemu wskazuje, że prawdopodobną przyczyną są zbyt wąskie ścieżki dla napięcia 5 V na płycie drukowanej, co prowadzi do zwiększenia rezystancji i ograniczenia przepływu prądu. W efekcie układ ładowania nie działa zgodnie z oczekiwaniami. Problem ten wynika z niedostatecznego uwzględnienia specyfikacji prądowo-napięciowej w projekcie PCB.

Drugim problemem było pominięcie połączenia jednego z pinów nadajnika LoRa RFM95 z resztą obwodu. Ten błąd konstrukcyjny uniemożliwił prawidłową pracę (problem ten można teoretycznie obejść poprzez odpowiednie napisanie kodu obsługującego RFM95 bez użycia pinu RESET, jednak w przypadku błędów transmisji moduł nie będzie w stanie powrócić do swoich ustawień fabrycznych). Błąd ten powstał na etapie projektowania schematu, gdzie prawdopodobnie przeoczono konieczność połączenia pinu z układem mikrokontrolera, bądź źle odczytano specyfikację komponentu.

Aby naprawić ten problem, zaplanowano prowizoryczne połączenie pinu do mikrokontrolera za pomocą ręcznego lutowania. Takie rozwiązanie, pozwala ono na weryfikację funkcjonalności układu oraz minimalizuje konieczność tworzenia nowej wersji płytki drukowanej.

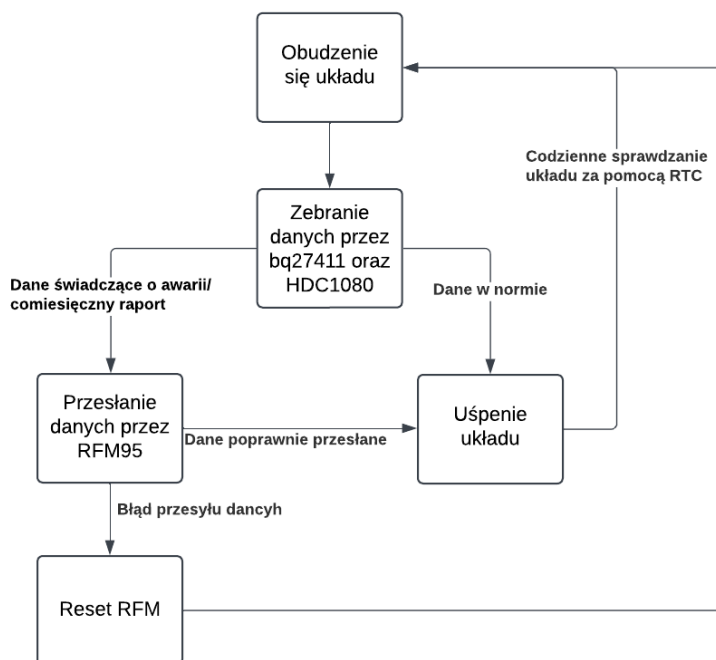


*Rysunek 26 Fotografia prototypu z połączonym zewnętrznym pinem RESET*

Na etapie projektowania urządzenia celowo przewidziano możliwość wystąpienia tego rodzaju problemów, dlatego obok mikrokontrolera STM zaprojektowano dodatkowe pady, które mogły służyć do późniejszego wykonania ręcznych modyfikacji lub napraw. Używając narzędzi lutowniczych, udało się przylutować brakujące połączenie. PAD2 połączony został z pinem RESET nadajnika. Użyto zwykłego, specjalnie podciętego, krótkiego kabla do płytek stykowych. Pomimo ograniczonej przestrzeni roboczej oraz konieczności zachowania szczególnej ostrożności, proces przebiegł pomyślnie. Po wykonaniu połączenia przeprowadzono testy funkcjonalne, które potwierdziły poprawną pracę elementu, wcześniej wykluczonego z działania.

#### 4.4.2 Zaktualizowane oprogramowanie

Kod został zaktualizowany w celu dostosowania funkcjonalności systemu, monitorowania baterii, komunikacji urządzenia za pomocą RFM95 oraz przechodzenia w stan uśpienia. Do stworzonego prototypu odpowiednio ustawiono piny, zadbane o poprawne ustawienie przerwań i ich priorytetów oraz sposoby komunikacji w sekcji „Pinout&Configuration”, natomiast w sekcji „Clock Configuration” ustawienia zegarów. Poniżej zilustrowano i opisano kluczowe fragmenty zaktualizowanego kodu. Architekturę kodu przedstawia poniższy diagram:



Rysunek 27 Schemat blokowy programu

Układ działa w sposób cykliczny, zaczynając od obudzenia się systemu, które jest inicjowane przez zegar czasu rzeczywistego (ang. real-time clock). RTC umożliwia codzienne sprawdzanie stanu systemu, co pozwala na regularne monitorowanie jego działania. Z racji tego, że w układzie nie jest używany zewnętrzny oscylator jako źródło zegara użyto wewnętrznego oscylatora RC. LSI (ang. Low-speed Internal) pozwala używać zegara o częstotliwości 37 kHz, co w przypadku używania czujnika jako urządzenia wczesnego ostrzegania jest jak najbardziej wystarczające. Po obudzeniu, układ zbiera dane z bq27411, oraz HDC1080. W przypadku, gdy dane są w normie, układ przechodzi w tryb uśpienia, aby zoptymalizować zużycie energii. Jeśli jednak dane wskazują na awarię lub wymagają stworzenia miesięcznego raportu, są one przesyłane dalej za pomocą modułu RFM95. Podczas przesyłania danych, jeśli transmisja zakończy się sukcesem, układ ponownie przechodzi w tryb uśpienia. Natomiast w przypadku wystąpienia błędu przesyłu, moduł RFM95 jest resetowany, a proces transmisji zostaje powtórzony. Całe oprogramowanie posiada odpowiednią strukturę, między innymi oddziela pliki nagłówkowe od źródłowych. W każdym z plików nagłówkowych zostały opisane funkcje oraz rejestry poszczególnych elementów układu.



Stworzono kod dla bq27411, by układ mógł informować o stanie naładowania baterii. Został skalibrowany do baterii używanej w urządzeniu. By odczyty były jak najbardziej miarodajne, tak samo jak w przypadku HDC1080 uśredniono je. Moduł RFM95 oraz mikrokontroler zostały wprowadzone w stan uśpienia, w trybie ciągłej pracy byłyby dużym obciążeniem prądowym dla układu co odbiło by się na jego energooszczędności. HDC1080 wchodzi w tryb uśpienia automatycznie po wykonanych pomiarach, natomiast bq27411 pozostawiono w trybie ciągłej pracy, ze względu na ryzyko zbyt dużej rozbieżności pomiarowej, mogącej wystąpić po wybudzeniu urządzenia. Częstotliwość zegara została ustawiona na 4,194 MHz, szybkość I<sup>2</sup>C na 100 kHz, natomiast szybkość transmisji danych SPI na 2,097 Mbit/s. Zmodyfikowano plik lora\_receiver.py pod zaktualizowane potrzeby projektu. W ten sposób nowszy plik RFM95\_receiver.py zastąpił swojego poprzednika. Pin GPIO, do którego podłączona została dioda LED, służył jako swego rodzaju breakpoint, podczas debuggowania kodu.

```
while (1)
{
    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

    HAL_RTC_GetTime(&hrtc, &currentTime, RTC_FORMAT_BIN);
    HAL_RTC_GetDate(&hrtc, &currentDate, RTC_FORMAT_BIN);

    currentTime.Hours = 17;
    currentTime.Minutes = 0;
    currentTime.Seconds = 0;

    float temperature = HDC1080_ReadTemperature();
    float humidity = HDC1080_AverageHumidity();
    float batteryLevel = bq27411_GetStableBatteryLevel();

    if (batteryLevel < 20.0 || humidity > 70.0) {
        char alertMessage[32];
        snprintf(alertMessage, sizeof(alertMessage), "T:%.1f, H:%.1f, B:%.0f", temperature,
humidity, batteryLevel);
        RFM95_SendMessage(alertMessage);
    }

    if (currentDate.Date == 1) {
        char monthlyReport[32];
        snprintf(monthlyReport, sizeof(monthlyReport), "T:%.1f, H:%.1f, B:%.0f", temperature,
humidity, batteryLevel);
        RFM95_SendMessage(monthlyReport);
    }

    RFM95_EnterSleepMode();
}
```

*Rysunek 28 Fragment kodu źródłowego main.c*

W pętli głównej funkcji `main()` system pracuje w trybie energooszczędnym, wybudzając się codziennie o ustalonej godzinie (17:00, wybrana została gdyż prawdopodobnie to właśnie o takiej porze ludzie wracają do domów i mogą zorientować się o zalaniu, bądź dysfunkcji systemu), dzięki zainicjowaniu RTC. Po wybudzeniu odczytywane są aktualny czas i data, a następnie wykonywane są pomiary temperatury, wilgotności i poziomu baterii. Jeśli wilgotność przekroczy 70 % lub poziom baterii spadnie poniżej 20 %, system generuje alert, który jest przesyłany za pomocą modułu RFM95. Dodatkowo, każdego 1. dnia miesiąca urządzenie wysyła miesięczny raport zawierający dane z pomiarów. Po zakończeniu zadań system przechodzi w głęboki tryb uśpienia. Dodatkowo skrócono wiadomość wysyłaną przez nadajnik. Skraca to znacząco czas operacji transmisji danych.

```

static void RFM95_WriteRegister(uint8_t reg, uint8_t value) {
    uint8_t buffer[2] = {reg | 0x80, value};
    HAL_GPIO_WritePin(rfmNssPort, rfmNssPin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(rfmSpi, buffer, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(rfmNssPort, rfmNssPin, GPIO_PIN_SET);
}

void RFM95_Init(SPI_HandleTypeDef *hspi, GPIO_TypeDef *nssPort, uint16_t nssPin, GPIO_TypeDef
*dio0Port, uint16_t dio0Pin) {
    rfmSpi = hspi;
    rfmNssPort = nssPort;
    rfmNssPin = nssPin;
    rfmDio0Port = dio0Port;
    rfmDio0Pin = dio0Pin;

    HAL_GPIO_WritePin(rfmNssPort, rfmNssPin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(rfmDio0Port, rfmDio0Pin, GPIO_PIN_RESET);
    HAL_Delay(10);
    HAL_GPIO_WritePin(rfmDio0Port, rfmDio0Pin, GPIO_PIN_SET);
    HAL_Delay(10);

    RFM95_EnterSleepMode();
    HAL_Delay(10);
    RFM95_EnterStandbyMode();
    RFM95_WriteRegister(0x0E, 0x80); // FIFO TX
    RFM95_WriteRegister(0x0F, 0x00); // FIFO RX

    // frequency
    uint32_t frf = (uint32_t)((double)DEFAULT_FREQUENCY / 61.03515625);
    RFM95_WriteRegister(REG_FRF_MSB, (frf >> 16) & 0xFF);
    RFM95_WriteRegister(REG_FRF_MID, (frf >> 8) & 0xFF);
    RFM95_WriteRegister(REG_FRF_LSB, frf & 0xFF);

    //Bandwidth, Coding Rate, Spreading Factor, CRC
    RFM95_WriteRegister(REG_MODEM_CONFIG1, DEFAULT_BANDWIDTH | DEFAULT_CODING_RATE);
    RFM95_WriteRegister(REG_MODEM_CONFIG2, DEFAULT_SPREADING_FACTOR | DEFAULT_CRC);
}

void RFM95_EnterSleepMode(void) {
    RFM95_WriteRegister(REG_OP_MODE, MODE_SLEEP);
    HAL_Delay(10);
}

void RFM95_EnterStandbyMode(void) {
    RFM95_WriteRegister(REG_OP_MODE, MODE_STDBY);
    HAL_Delay(10);
}

void RFM95_SetPayloadLength(uint8_t length) {
    RFM95_EnterStandbyMode();
    RFM95_WriteRegister(REG_PAYLOAD_LENGTH, length);
}

void RFM95_SendMessage(const char *message) {
    uint8_t length = strlen(message);
    RFM95_EnterStandbyMode();
    RFM95_WriteRegister(0x0D, 0x80); // fifo * -> tx

    for (uint8_t i = 0; i < length; i++) {
        RFM95_WriteRegister(REG_FIFO, message[i]);
    }

    RFM95_SetPayloadLength(length);
    RFM95_WriteRegister(REG_OP_MODE, MODE_TX);

    // tx / error
    uint32_t startTick = HAL_GetTick();
    while (HAL_GPIO_ReadPin(rfmDio0Port, rfmDio0Pin) == GPIO_PIN_RESET) {
        if (HAL_GetTick() - startTick > TX_TIMEOUT) {
            // Timeout
            RFM95_HandleError();
            return;
        }
    }

    RFM95_WriteRegister(REG_IRQ_FLAGS, 0x08); //clr txdone flag
}

void RFM95_HandleError(void) {
    // proba resetu modulu
    HAL_GPIO_WritePin(rfmDio0Port, rfmDio0Pin, GPIO_PIN_RESET);
    HAL_Delay(10);
    HAL_GPIO_WritePin(rfmDio0Port, rfmDio0Pin, GPIO_PIN_SET);
    HAL_Delay(10);

    RFM95_EnterSleepMode();
    HAL_Delay(10);
    RFM95_EnterStandbyMode();

    uint32_t frf = (uint32_t)((double)DEFAULT_FREQUENCY / 61.03515625);
    RFM95_WriteRegister(REG_FRF_MSB, (frf >> 16) & 0xFF);
    RFM95_WriteRegister(REG_FRF_MID, (frf >> 8) & 0xFF);
    RFM95_WriteRegister(REG_FRF_LSB, frf & 0xFF);

    RFM95_WriteRegister(REG_MODEM_CONFIG1, DEFAULT_BANDWIDTH | DEFAULT_CODING_RATE);
    RFM95_WriteRegister(REG_MODEM_CONFIG2, DEFAULT_SPREADING_FACTOR | DEFAULT_CRC);
}

```

Rysunek 29 Fragment kodu źródłowego rfm95.c

Program `rfm95.c` powstał na bazie pliku `loratesting.c`. Podstawową funkcją jest `RFM95_WriteRegister`, która pozwala na zapisanie wartości do dowolnego rejestru modułu. Działanie tej funkcji polega na aktywowaniu komunikacji przez ustawienie pinu NSS w stan niski, wysłaniu adresu rejestru i wartości przez SPI, a następnie zakończeniu transmisji przez ustawienie pinu NSS w stan wysoki. Funkcja `RFM95_Init` inicjalizuje moduł, przypisuje odpowiednie piny GPIO i SPI i resetuje moduł za pomocą pinów RESET i DIO. Po wprowadzeniu modułu w tryb uśpienia i gotowości (standby), konfiguracja obejmuje ustawienie częstotliwości pracy (poprzez obliczenie wartości dla rejestrów `FRF_MSB`, `FRF_MID` i `FRF_LSB`) oraz parametrów transmisji, takich jak pasmo, szybkość kodowania, spreading factor i CRC. Te parametry są zapisywane do rejestrów `REG_MODEM_CONFIG1` i `REG_MODEM_CONFIG2`. Co ważne, przy tych ustawieniach urządzenie działa i nadaje (moc wyjściowa to 13 dbm) zgodnie z prawem [29]. Zarządzanie trybami pracy modułu realizowane jest poprzez funkcje `RFM95_EnterSleepMode` i `RFM95_EnterStandbyMode`. `RFM95_EnterSleepMode` przełącza moduł w tryb uśpienia poprzez zapis odpowiedniej wartości do rejestru `REG_OP_MODE`, a następnie dodaje niewielkie opóźnienie dla stabilizacji. `RFM95_EnterStandbyMode` wprowadza moduł w tryb gotowości, który pozwala na natychmiastową transmisję lub odbiór danych. Do wysyłania danych służy funkcja `RFM95_SendMessage`. Najpierw przełącza moduł w tryb standby (jest to podyktowane wymogiem producenta, który informuje, że ustawianie parametrów nadajnika oraz danych do wysłania powinno odbyć się w trybie uśpienia, bądź czuwania [12]), a następnie ustawia FIFO w tryb nadawania. Dane są przesyłane w pętli, zapisując je kolejno do rejestru `REG_FIFO`. Funkcja ustawia również długość wiadomości za pomocą `RFM95_SetPayloadLength`, po czym przełącza moduł w tryb nadawania (`MODE_TX`). Stan transmisji jest monitorowany poprzez odczyt pinu DIO0, który sygnalizuje zakończenie transmisji. Jeśli transmisja trwa zbyt długo i przekracza zdefiniowany czas `TX_TIMEOUT` (3 sekundy), wywoływana jest funkcja obsługi błędów `RFM95_HandleError`. Na początku, podczas pierwszych testów, czas przesyłu danych napisany był jawnie, jednakże wtedy często pojawiały się błędy, gdyż mimo użycia wzoru na czas transmisji (Time on Air [12]), czas ten różnił się, toteż zdecydowano się na użycie licznika. `RFM95_HandleError` obsługuje sytuacje awaryjne, resetując moduł za pomocą pinów RESET i DIO. Po wprowadzeniu opóźnienia stabilizacyjnego przywracane są podstawowe ustawienia modułu, takie jak częstotliwość i parametry transmisji. Podczas testowania układu w sytuacjach, gdy wystąpiły błędy transmisji, zauważono istotny problem: w przypadku awarii modułu RFM95 urządzenie nie jest w stanie fizycznie powiadomić użytkownika o swoim niesprawnym działaniu. W obecnej konfiguracji brak jest mechanizmu, który mógłby sygnalizować takie sytuacje. Jeśli urządzenie przestanie wysyłać dane, użytkownik nie zostanie o tym poinformowany, co może prowadzić do opóźnionego wykrycia problemu. W przyszłych iteracjach prototypu rozważa się dodanie brzęczyka, który umożliwiłby bezpośrednie, dźwiękowe powiadomienie o błędzie transmisji. Wykorzystanie diody LED zostało odrzucone ze względu na fakt, że urządzenie jest umieszczone w trudno dostępnym miejscu, gdzie sygnalizacja świetlna nie byłaby widoczna. W obecnej sytuacji użytkownik może zorientować się o niesprawności układu dopiero po miesiącu, gdy zauważy brak zaktualizowanego raportu w aplikacji Home Assistant. Dodanie brzęczyka mogłoby znacząco poprawić niezawodność systemu oraz ułatwić szybkie wykrywanie usterek.



```

void bq27411_ConfigureBattery(void) {
    uint8_t designCapacity[3] = {0x3C, 0xD0, 0x07}; // 2000mAh
    HAL_I2C_Master_Transmit(bqI2c, BQ27411_ADDRESS << 1, designCapacity, 3, HAL_MAX_DELAY);
}

float bq27411_GetStableBatteryLevel(void) {
    uint8_t measurements[10];
    for (int i = 0; i < 10; i++) {
        uint8_t buffer[2];
        HAL_I2C_Mem_Read(bqI2c, BQ27411_ADDRESS << 1, 0x2C, I2C_MEMADD_SIZE_8BIT, buffer, 2,
        HAL_MAX_DELAY);
        uint16_t level = (buffer[0] << 8) | buffer[1];
        measurements[i] = level / 256;
        HAL_Delay(50);
    }
    uint16_t sum = 0;
    for (int i = 5; i < 10; i++) {
        sum += measurements[i];
    }
    return sum / 5.0f;
}

```

*Rysunek 30 Fragment kodu źródłowego bq27411.c*

Funkcja `bq27411_ConfigureBattery` kalibruje pojemność baterii na 2000 mAh, zapisując odpowiednie dane do rejestru `Design Capacity` bq27411. Funkcja `bq27411_GetStableBatteryLevel` działa analogicznie do `HDC1080`, tylko że pobiera dane o SoC.

Podsumowując, procesy wybudzania, zbierania i przesyłania danych, a także resetowania układu w przypadku wystąpienia błędów, tworzą kompletny cykl pracy systemu, zapewniając jego efektywne i niezawodne działanie.

#### 4.4.3 Testy prototypu

Niestety nie udało się przeprowadzić testów ładowania baterii oraz pracy na zewnętrznym zasilaniu, gdyż układ okazał się niesprawny. W ramach następnej iteracji prototypowania, ścieżkę 0,3 mm trzeba pogrubić o 0,2 mm lub o nawet 0,3 mm. Wpłynie to na rozstaw elementów, ale na płycie aktualnych rozmiarów, jest to akceptowalne. Dodatkowo zapobiegawczo można zwiększyć minimalne odstępy między ścieżkami.

Przeprowadzono następujące testy:

- Test i bilans energetyczny został przeprowadzony zgodnie ze scenariuszem, w którym przez 14 dni, raz dziennie, urządzenie robiło pomiary i przesyłało dane.

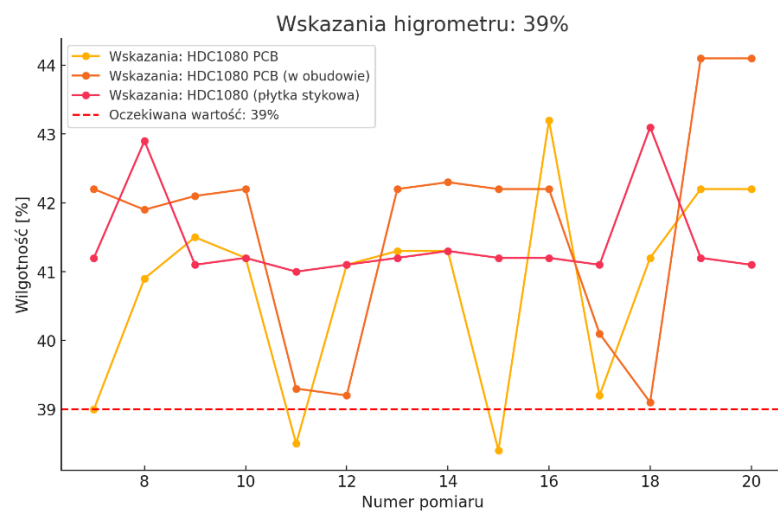
Tabela 2 Zestawienie prądowe elementów

| Komponent     | Prąd w trybie pracy             | Prąd w trybie uśpienia |
|---------------|---------------------------------|------------------------|
| RFM95         | do ok. 90 mA w trybie wysyłania | 0,2 $\mu$ A            |
| HDC1080       | 1,3 $\mu$ A                     | 100 nA                 |
| bq27411       | 93 $\mu$ A                      | 21 $\mu$ A             |
| STM32L071KBU6 | ok. 3 mA                        | 0,5 $\mu$ A            |

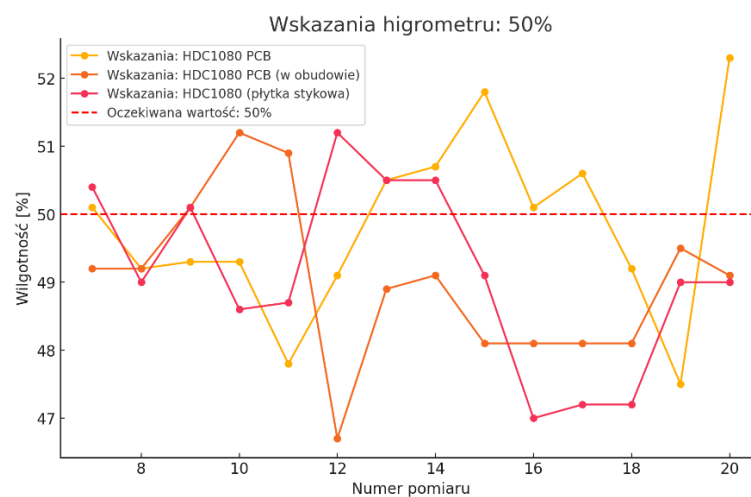
Prąd regulatora liniowego TPS7A02 jest praktycznie znikomy, wynosi 25 nA. Na linii regulatora znajdują się również dwa rezystory podciągające o wartościach 15 k $\Omega$ , co skutkuje wysokim poborem prądu w postaci łącznie 440  $\mu$ A. Bilans został oszacowany pomijając minimalne wartości prądu w stanie spoczynku, gdyż w całym rachunku, byłyby zaledwie minimalnym odchyleniem. Dzielne zużycie RFM95 oszacowano na 0.025 mAh, na co składa się 90 mA \* ~150 ms, czyli czas wysyłania. Komponent bq27411 w 24 godziny przy prądzie 93  $\mu$ A, wykorzystywać będzie około 2,2 mAh. Elementy kolejno HDC1080, STM32L071KBU6, TPS7A02 dziennie będą kolejno zużywały około: 0.002 mAh, 0.01 mAh oraz 0.00006 mAh. Największym problemem jest para rezystorów podciągających konsumująca około 10.5 mAh dziennie. Zatem łączne dziennie sumaryczne zużycie energii szacowane jest na około: 12,8 mAh. Wartość ta pomnożona 14 razy (dwa tygodnie) wynosi, 179,2 mAh, co implikuje szacowaną wartość SoC na 92% po wykonaniu scenariusza. Monitor stanu baterii po tym teście wskazał wartość 88%, co poniekąd odbiega od założenia, natomiast jest akceptowalne. Przy takiej konfiguracji urządzenie mogłoby działać przez około 4 miesiące bez konieczności ładowania, bądź wymiany baterii, co jest dobrym wynikiem. Przy przyjętym scenariuszu początkowym, czyli wysyłaniem danych raz na miesiąc i codziennym sprawdzaniu poprawnego działania czujnika, byłby on w stanie pracować nieco dłużej (przez około 5 miesięcy). Niezgodności w bilansie mogą wynikać z niedoskonałości biblioteki HAL (czas teoretyczny w trybie aktywnej pracy może różnić się od faktycznego), oraz przez prawdopodobny wyciek prądu przy bq24078. W fazie testu nie zostały wyłączone nieużywane piny mikrokontrolera.

- Testy czujnika wilgotności

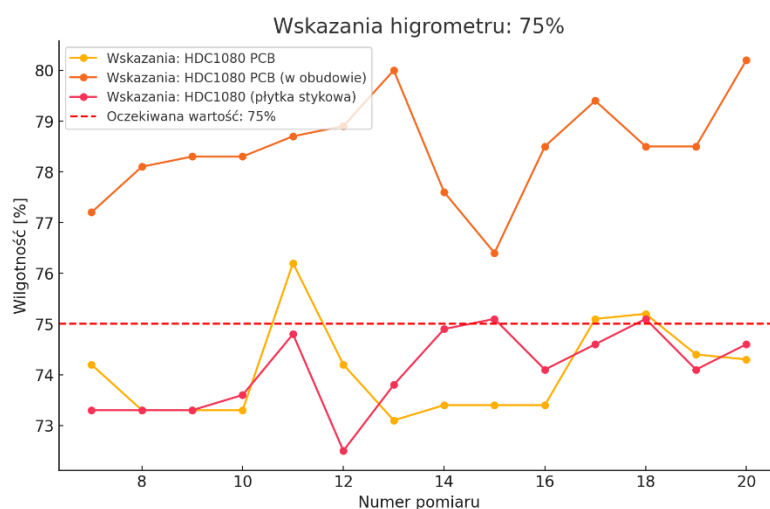
Wartości przesyłane przez czujnik wilgotności sprawdziły się, nawet gdy został opakowany on w prowizoryczne pudełko ochronne. Został przetestowany obok HDC1080 z wcześniejszych testów. Postanowiono zmienić zakres ostrzegania o ewentualnej wilgotności na 70 %, gdyż jest to wartość bardziej uniwersalna (trzeba brać pod uwagę odległość czujnika od miejsca ewentualnej awarii).



Rysunek 31 Wyniki I scenariusza testowego HDC1080 PCB



Rysunek 32 Wyniki II scenariusza testowego HDC1080 PCB



Rysunek 33 Wyniki III scenariusza testowego HDC1080 PCB

Pominięte zostało pierwsze 7 pomiarów każdego z nich, ze względu na ich kalibrowanie się. Wyniki pokrywają się z rzeczywistością, co potwierdza poprawne działanie HDC1080. Przy pomiarze z oczekiwaną wartością 75 % wilgotności HDC1080 w obudowie wskazywał wyższe wartości. Jest to spowodowane trudnością utrzymania jednolitego środowiska testowego, przy wyższych wskazaniach wilgotności.

- Testy transmisji

Z wcześniejszego testu energetycznego, gdy urządzenie leżało w pokoju w warunkach pokojowych, dane zostały odbierane przez RFM95 podłączony do Raspberry Pi 4 bez problemu. Następnie urządzenie zostało zaimplementowane w prowizoryczną obudowę i ustawione w tryb ciągłego nadawania, by na żywo w zależności od położenia sprawdzać czy sygnał jest odbierany. Powtórzono podobne scenariusze testów takich jak przy testowaniu modularnym LoRa. Sygnał był prawidłowo przesyłany, co ważne nawet w miejscu pokrewnym jego typowym zastosowaniu; w skrytce na liczniki wody.

#### 4.4.4 Możliwe optymalizacje i dalsze kroki

Rezystory podciągające są kluczowym problemem projektu. Stanowią główną część konsumpcji prądu. Te znaczące straty energetyczne, mogą zostać ominięte przez spróbowanie różnych sposobów w następnych iteracjach prototypu. Pierwszym, najlogiczniejszym jest sterowanie nimi za pomocą tranzystorów. Budując odpowiedni układ z tranzystorów, mogłyby one odciągać rezystory od zasilania, gdy szyna magistrali I<sup>2</sup>C jest nieużywana. W ten sposób z czujnika energooszczędnego, można pokusić by się było o czujnik ultra energooszczędny. Zwiększa to co prawda skomplikowanie projektu i ogranicza miejsce na płytce, ale te wady są znacząco przyćmiewane przez korzyści dodania MOSFET-ów. Drugą możliwością jest całkowite pozbycie się zewnętrznych rezystorów podciągających, a zamiast nich próba użycia rezystorów wewnętrznych mikroprocesora. Metoda, ta jest bardzo ciężka do wykonania, gdyż rezystory te mają zazwyczaj wartości od 40 do 60 kΩ. W takim wypadku ścieżki musiałyby zostać maksymalnie skrócone, by ograniczyć ich wpływ pojemności i umożliwić pracę magistrali I<sup>2</sup>C.

Aby zmniejszyć zużycie energii, można wprowadzić układ bq27411 w stan uśpienia. Układ ten również zabierał około 2 mAh każdego dnia. W kolejnej iteracji prototypu przed testami należy zaktualizować kod i wprowadzić komponent w tryb deepsleep.

Kolejną rzeczą jest konieczne poprawne zaimplementowanie układu bq24078. Wpływa to na komfort użytkowania urządzenia, oraz na jego energooszczędność. Dodatkowo w urządzeniu kolejnej iteracji, zwiększyć szybkość ładowania urządzenia. Sam interfejs USB-C mógłby zostać użyty w celu komunikacji z komputerem, przy desktopowej aplikacji, w której to możliwe byłoby ustawienie jednego z trybów czujnika.

Kolejnym prostym sposobem na zwiększenie czasu pracy urządzenia jest znalezienie baterii o większej pojemności, w podobnych rozmiarach. Baterie typu Li-Ion są szeroko popularne, a ich ceny przy mniejszych pojemnościach nie różnią się znacząco.

W przyszłych iteracjach prototypu trzeba rozważyć zastosowanie np. brzęczyka jako elementu sygnalizacyjnego, który umożliwi informowanie użytkownika o błędach działania układu w sposób sprzętowy. Takie rozwiązanie pozwoliłoby na niezależne od oprogramowania powiadamianie o problemach, co zwiększyłoby niezawodność systemu.

Kolejną bardzo ważną rzeczą jest obudowa. Odpowiednia obudowa zapewniałaby bezpieczeństwo urządzenia. Podczas kolejnej iteracji, pierwsze urządzenie można zalać, by przeprowadzić testy trwałości.

W systemie Home Assistant, w już istniejącym rozwiązaniu, ustawić inteligentną lampkę/diodę, która zintegrowana będzie z czujnikiem (np. w razie zalania, bądź niesprawności układu zacznie mrugać).

## 5 Podsumowanie

Niniejsze podsumowanie pracy inżynierskiej, której tematem wykonanie było energooszczędne czujnika wilgotności dla inteligentnego domu, stanowi jej przegląd wyników i osiągnięć. Dzięki starannemu dobraniu komponentów układu oraz stworzeniu hardware'u jak i software'u ostateczny prototyp okazał się być prawie w pełni sprawnym urządzeniem. Ładowanie poprzez interfejs USB-C oraz praca na zasilaniu zewnętrznym układu okazała się być niemożliwa, natomiast precyzja pomiaru, zasięg oraz penetracja transmitowanego sygnału są na satysfakcjonującym poziomie. Wykorzystanie technologii LoRa, okazało się kluczowe dla nienagannej transmisji z każdego miejsca w standardowym gospodarstwie domowym. Bardzo ważnym parametrem jest energooszczędność, która dzięki opracowaniu powyższego prototypu, może zostać bardzo dobrze zoptymalizowana, czyniąc urządzenie ultra-energooszczędnym. Układ wpisuje się w ramy Internetu Rzeczy, będąc w stanie być jego integralną częścią. Urządzenie idealnie wpisałoby się w niszowy rynek takowych czujników, w momencie szybkiego rozwoju IoT. Trzeba zwrócić należytą uwagę, na koszt użytych elementów. Cena wszystkich elementów to około 15 USD (stan na 9 listopada 2024), co z przyszłym rozwojem może uczynić go bardzo konkurencyjnym urządzeniem rynkowym.

## Bibliografia

- [1] R. Kamal „INTERNET OF THINGS – Architecture and Design Principles”, 2017
- [2] R. Bogacz, B. Krupanek „WĘZŁY KOŃCOWE SYSTEMÓW INTERNETU RZECZY”, L Międzyuczelniana Konferencja Metrologów MKM 2018, Szczecin – Kopenhaga, 10-12 września 2018
- [3] Device Authority „Unpacking IoT Architecture: Layers and Components Explained”, pozyskano 15 stycznia 2025, z <https://deviceauthority.com/unpacking-iot-architecture-layers-and-components-explained/#:~:text=IoT%20architecture%20is%20an%20organised,processing%20and%20User%20interface%2Fapplication> b.d.
- [4] T. Cooklev „Wireless Communication Standards: A Study of IEEE 802.11, 802.15, 802.16”, 2004
- [5] 300Gospodarka „Czym jest technologia NB-IoT i jak usprawni działanie firmy”, pozyskano 16 stycznia 2025, z <https://300gospodarka.pl/material-partnera/czym-jest-technologia-nb-iot-i-jak-usprawni-dzialanie-firmy>, 12 lipca 2024
- [6] GSMA „5G IoT”, pozyskano 16 stycznia 2025, z <https://www.gsma.com/solutions-and-impact/technologies/internet-of-things/5g-iot/> b.d.
- [7] Interline „What technology behind LoRa frequency”, pozyskano 19 stycznia 2025, z <https://interline.pl/pl/Information-and-Tips/What-Technology-Behind-LoRa-Frequency>, b.d.
- [8] ETSI „TR 103 526 – V1.1.1 - System Reference Document (SRdoc); Technical characteristics for Low Power Wide Area Networks Chirp Spread Spectrum (LPWAN-CSS) operating in the UHF spectrum below 1 GHz”, kwiecień 2018.
- [9] STMicroelectronics „STM32L071KB – Microcontroller” pozyskano 26 stycznia 2025 z <https://www.st.com/en/microcontrollers-microprocessors/stm32l071kb.html>
- [10] Texas Instruments „MSP430 Microcontrollers” pozyskano 26 stycznia 2025 z <https://www.ti.com/microcontrollers-mcus-processors/msp430-microcontrollers/products.html>
- [11] Microchip Technology „SAM 32-Bit MCU's” pozyskano 26 stycznia 2025 z <https://www.microchip.com/en-us/products/microcontrollers-and-microprocessors/32-bit-mcus/sam-32-bit-mcus/sam-l>
- [12] SparkFun „HOPERF RFM95/96/97/98W Datasheet” pozyskano 26 stycznia 2025 z [https://cdn.sparkfun.com/assets/learn\\_tutorials/8/0/4/RFM95\\_96\\_97\\_98W.pdf](https://cdn.sparkfun.com/assets/learn_tutorials/8/0/4/RFM95_96_97_98W.pdf)
- [13] Semtech „SX1276 – LoRa Transceiver Datasheet” pozyskano 26 stycznia 2025 z <https://www.semtech.com/products/wireless-rf/lora-connect/sx1276>
- [14] Texas Instruments „HDC1080 – Temperature and Humidity Sensor Datasheet” pozyskano 26 stycznia 2025 z <https://www.ti.com/lit/ds/symlink/hdc1080.pdf?ts=1737839133279>
- [15] Sensirion „SHT3x – Humidity and Temperature Sensor Datasheet” pozyskano 26 stycznia 2025 z [https://sensirion.com/media/documents/213E6A3B/63A5A569/Datasheet\\_SHT3x\\_DIS.pdf](https://sensirion.com/media/documents/213E6A3B/63A5A569/Datasheet_SHT3x_DIS.pdf)
- [16] Adafruit „Aosong AM2320 - Datasheet” pozyskano 26 stycznia z <https://cdn-shop.adafruit.com/product-files/3721/AM2320.pdf>
- [17] Adafruit „TE CONNECTIVITY HTU21D - Datasheet” pozyskano 26 stycznia 2025 z [https://cdn-shop.adafruit.com/datasheets/1899\\_HTU21D.pdf](https://cdn-shop.adafruit.com/datasheets/1899_HTU21D.pdf)

- [18] Texas Instruments „bq24078 – Charger Datasheet” pozyskano 26 stycznia 2025 z <https://www.ti.com/lit/gpn/bq24078>
- [19] Texas Instruments „bq27411-g1 – Fuel Gauge Datasheet” pozyskano 26 stycznia 2025 z <https://www.ti.com/lit/gpn/bq27411-g1>
- [20] Texas Instruments „TPS7A02 – Low-Dropout Regulator Datasheet” pozyskano 26 stycznia 2025 z <https://www.ti.com/lit/gpn/tps7a02>
- [21] PCBWay Custom PCB Prototype Manufacturer pozyskano 26 stycznia 2025 z <https://www.pcbway.com>
- [22] STMicroelectronics „STM32F4 Discovery Kit Datasheet” pozyskano 26 stycznia 2025 z [https://www.st.com/resource/en/data\\_brief/stm32f4discovery.pdf](https://www.st.com/resource/en/data_brief/stm32f4discovery.pdf)
- [23] Jameco „UNO R3 CH340” pozyskano 26 stycznia 2025 z [https://www.jameco.com/Jameco/Products/ProdDS/2297727.pdf?srsId=AfmBOoq1kj\\_n8eOh9S9Urn-q4uMZRBv5F2uavtXBSiY9M9kM6y4drw8L](https://www.jameco.com/Jameco/Products/ProdDS/2297727.pdf?srsId=AfmBOoq1kj_n8eOh9S9Urn-q4uMZRBv5F2uavtXBSiY9M9kM6y4drw8L)
- [24] Mouser Electronics „Kuongshun Electronic DHT11 – Technical Data Sheet” pozyskano 26 stycznia 2025 z [https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf?srsId=AfmBOoqATvrazAlek3p5lm0v4\\_sHN4aDZQt6av4\\_-\\_plyDOqY-M2Jdyx](https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf?srsId=AfmBOoqATvrazAlek3p5lm0v4_sHN4aDZQt6av4_-_plyDOqY-M2Jdyx)
- [25] Velcon „SAMYOUNG SYH-2R Datasheet” pozyskano 26 stycznia 2025 z <https://www.velcom.com.pl/pdf/440.pdf>
- [26] Xiaomi „Mi temperature and Humidity Monitor 2 – User Manual” pozyskano 26 stycznia 2025 z <https://mimarkt.pl/images/companies/1/Instrukcje/Mi%20Temperature%20and%20Humidity%20Monitor%202.pdf>
- [27] Raspberry Pi Trading „Raspberry Pi 4 Product Brief” pozyskano 26 stycznia 2025 z <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-product-brief.pdf>
- [28] HiveMQ „MQTT Essentials: The Ultimate Guide to the MQTT Protocol for IoT Messaging” Wersja 2.0. b.d
- [29] ROZPORZĄDZENIE MINISTRA CYFRYZACJI 1, 2 z dnia 9 lutego 2022 r. w sprawie urządzeń radiowych nadawczych lub nadawczo-odbiorczych, które mogą być używane bez pozwolenia radiowego.



## Spis rysunków

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| Rysunek 1 Zobrazowanie działania CSS [7] .....                                    | 16 |
| Rysunek 2 Schemat blokowy układu.....                                             | 20 |
| Rysunek 3 Część zasilania .....                                                   | 21 |
| Rysunek 4 Szczegółowy schemat pierwszej sekcji części zasilania.....              | 22 |
| Rysunek 5 Szczegółowy schemat drugiej sekcji części zasilania.....                | 23 |
| Rysunek 6 Część cyfrowa.....                                                      | 25 |
| Rysunek 7 Szczegółowy schemat pierwszej sekcji części cyfrowej.....               | 26 |
| Rysunek 8 Szczegółowy schemat drugiej sekcji części cyfrowej.....                 | 27 |
| Rysunek 9 Szczegółowy schemat trzeciej sekcji części cyfrowej .....               | 28 |
| Rysunek 10 Dolna warstwa PCB- część zasilania .....                               | 30 |
| Rysunek 11 Górna warstwa PCB- część cyfrowa.....                                  | 30 |
| Rysunek 12 Fotografia prototypu z baterią.....                                    | 31 |
| Rysunek 13 Fotografia prototypu - górna warstwa .....                             | 31 |
| Rysunek 14 Stanowisko pomiarowe czujników .....                                   | 33 |
| Rysunek 15 Fragment kodu źródłowego hdc1080testing.c .....                        | 34 |
| Rysunek 16 Wyniki I scenariusza testowego .....                                   | 35 |
| Rysunek 17 Wyniki II scenariusza testowego.....                                   | 35 |
| Rysunek 18 Wyniki III scenariusza testowego.....                                  | 36 |
| Rysunek 19 Fragment kodu źródłowego loratesting.c.....                            | 37 |
| Rysunek 20 Stanowisko testowe LoRa .....                                          | 38 |
| Rysunek 21 Schemat komunikacji MQTT [28].....                                     | 39 |
| Rysunek 22 Fragment kodu źródłowego lora_receiver.py .....                        | 39 |
| Rysunek 23 Kod źródłowy configuration.yaml (nasłuch Home Assistant) .....         | 40 |
| Rysunek 24 Fragment panelu głównego Home Assistant z danymi .....                 | 40 |
| Rysunek 25 Fotografia prototypu - dolna warstwa wraz z debuggerem ST-LINK V2..... | 41 |
| Rysunek 26 Fotografia prototypu z połączonym zewnętrznym pinem RESET .....        | 42 |
| Rysunek 27 Schemat blokowy programu .....                                         | 43 |
| Rysunek 28 Fragment kodu źródłowego main.c.....                                   | 44 |
| Rysunek 29 Fragment kodu źródłowego rfm95.c .....                                 | 45 |
| Rysunek 30 Fragment kodu źródłowego bq27411.c.....                                | 47 |
| Rysunek 31 Wyniki I scenariusza testowego HDC1080 PCB.....                        | 49 |
| Rysunek 32 Wyniki II scenariusza testowego HDC1080 PCB.....                       | 49 |
| Rysunek 33 Wyniki III scenariusza testowego HDC1080 PCB.....                      | 49 |

## Spis tabel

|                                                           |    |
|-----------------------------------------------------------|----|
| Tabela 1 Porównanie technologii komunikacyjnych IoT ..... | 14 |
| Tabela 2 Zestawienie prądowe elementów .....              | 48 |

